

Table 36 – continued from previous page

Language Feature	C++17 Proposal	NVCC/CUDA Toolkit 11.x
<code>[[fallthrough]]</code> attribute	P0188R1	✓
<code>[[nodiscard]]</code> attribute	P0189R1	✓
<code>[[maybe_unused]]</code> attribute	P0212R1	✓
Extension to aggregate initialization	P0017R1	✓
Wording for <code>constexpr</code> lambda	P0170R1	✓
Unary Folds and Empty Parameter Packs	P0036R0	✓
Generalizing the Range-Based For Loop	P0184R0	✓
Lambda capture of <code>*this</code> by Value	P0018R3	✓
Construction Rules for <code>enum class</code> variables	P0138R2	✓
Hexadecimal floating literals for C++	P0245R1	✓
Dynamic memory allocation for over-aligned data	P0035R4	✓
Guaranteed copy elision	P0135R1	✓
Refining Expression Evaluation Order for Idiomatic C++	P0145R3	✓
<code>constexpr if</code>	P0292R2	✓
Selection statements with initializer	P0305R1	✓
Template argument deduction for class templates	P0091R3 P0512R0	✓
Declaring non-type template parameters with <code>auto</code>	P0127R2	✓
Using attribute namespaces without repetition	P0028R4	✓
Ignoring unsupported non-standard attributes	P0283R2	✓
<i>Structured bindings</i>	P0217R3	✓
Remove Deprecated Use of the <code>register</code> Keyword	P0001R1	✓
Remove Deprecated <code>operator++(bool)</code>	P0002R1	✓
Make exception specifications be part of the type system	P0012R1	✓
<code>__has_include</code> for C++17	P0061R1	✓
Rewording inheriting constructors (core issue 1941 et al)	P0136R1	✓
<i>Inline variables</i>	P0386R2	✓
DR 150, Matching of template template arguments	P0522R0	✓
Removing dynamic exception specifications	P0003R5	✓
Pack expansions in using-declarations	P0195R2	✓
A byte type definition	P0298R0	✓

continues on next page

Table 36 – continued from previous page

Language Feature	C++17 Proposal	NVCC/CUDA Toolkit 11.x
DR 727, In-class explicit instantiations	CWG727	✓

5.3.4. C++20 Language Features

GCC version ≥ 10.0 , Clang version ≥ 10.0 , Microsoft Visual Studio ≥ 2022 , and nvc++ version ≥ 20.7 .

Note

Entries prefixed with “DR:” are Defect Report resolutions. They correct the standard and apply to earlier C++ standard modes (e.g., C++17) as well; they are listed here for completeness and are not specific to C++20.

Table 37: C++20 Language Features Supported by NVCC for device code

Language Feature	C++20 Proposal	NVCC/CUDA Toolkit 12.x
Default member initializers for bit-fields	P0683R1	✓
Fixing const-qualified pointers to members	P0704R1	✓
Allow lambda capture [=, this]	P0409R2	✓
__VA_OPT__ for preprocessor comma elision	P0306R4 P1042R1	✓
Designated initializers	P0329R4	✓
Familiar template syntax for generic lambdas	P0428R2	✓
List deduction of vector	P0702R1	✓
Concepts	P0734R0 P0857R0 P1084R2 P1141R2 P0848R3 P1616R1 P1452R2 P1972R0 P1980R0 P2092R0 P2103R0 P2113R0	✓
Range-based for statements with initializer	P0614R1	✓
Simplifying implicit lambda capture	P0588R1	✓
ADL and function templates that are not visible	P0846R0	✓
const mismatch with defaulted copy constructor	P0641R2	✓
Less eager instantiation of constexpr functions	P0859R0	✓

continues on next page

Table 37 – continued from previous page

Language Feature	C++20 Proposal	NVCC/CUDA Toolkit 12.x
<i>Consistent comparison</i> (operator<=>)	P0515R3 P0905R1 P1120R0 P1185R2 P1186R3 P1630R1 P1946R0 P1959R0 P2002R1 P2085R0	✓
Access checking on specializations	P0692R1	✓
Default constructible and assignable stateless lambdas	P0624R2	✓
Lambdas in unevaluated contexts	P0315R4	✓
Language support for empty objects	P0840R2	✓
Relaxing the range-for loop customization point finding rules	P0962R1	✓
<i>Allow structured bindings to accessible members</i>	P0969R0	✓
Relaxing the structured bindings customization point finding rules	P0961R1	✓
Down with typename!	P0634R3	✓
Allow pack expansion in lambda init-capture	P0780R2 P2095R0	✓
Proposed wording for likely and unlikely attributes	P0479R5	✓
Deprecate implicit capture of this via [=]	P0806R2	✓
Class Types in Non-Type Template Parameters	P0732R2	✓
Inconsistencies with non-type template parameters	P1907R1	✓
Atomic Compare-and-Exchange with Padding Bits	P0528R3	✓
Efficient sized delete for variable sized classes	P0722R3	✓
Allowing Virtual Function Calls in Constant Expressions	P1064R0	✓
Prohibit aggregates with user-declared constructors	P1008R1	✓
explicit(bool)	P0892R2	✓
Signed integers are two's complement	P1236R1	✓
char8_t	P0482R6	✓
<i>Immediate functions</i> (constexpr)	P1073R3 P1937R2	✓
std::is_constant_evaluated	P0595R2	✓
Nested inline namespaces	P1094R2	✓
Relaxations of constexpr restrictions	P1002R1 P1327R1 P1330R0 P1331R2 P1668R1 P0784R7	✓

continues on next page

Table 37 – continued from previous page

Language Feature	C++20 Proposal	NVCC/CUDA Toolkit 12.x
Feature test macros	P0941R2	✓
Modules	P1103R3 P1766R1 P1811R0 P1703R1 P1874R1 P1979R0 P1779R3 P1857R3 P2115R0 P1815R2	×
Coroutines	P0912R5	×
Parenthesized initialization of aggregates	P0960R3 P1975R0	✓
DR: array size deduction in new-expression	P1009R2	✓
DR: Converting from T* to bool should be considered narrowing	P1957R2	✓
Stronger Unicode requirements	P1041R4 P1139R2	✓
Structured binding extensions	P1091R3 P1381R1	✓
Deprecate a[b, c]	P1161R3	✓
Deprecating some uses of volatile	P1152R4	✓
[[nodiscard("with reason")]]	P1301R4	✓
using enum	P1099R5	✓
Class template argument deduction for aggregates	P1816R0 P2082R1	✓
Class template argument deduction for alias templates	P1814R0	✓
Permit conversions to arrays of unknown bound	P0388R4	✓
constexpr	P1143R2	✓
Layout-compatibility and Pointer-interconvertibility Traits	P0466R5	✓
DR: Checking for abstract class types	P0929R2	✓
DR: More implicit moves	P1825R0	✓
DR: Pseudo-destructors end object lifetimes	P0593R6	✓

5.3.5. C++23 Language Features

GCC version ≥ 14.0 , Clang version ≥ 18.0 , Microsoft Visual Studio (Not Supported), and nvcc++ version ≥ 24.3 .

Note

Entries prefixed with “DR:” are Defect Report resolutions. They correct the standard and apply to earlier C++ standard modes (e.g., C++17, C++20) as well; they are listed here for completeness and

are not specific to C++23.

Note

N/A in the NVCC column indicates that the feature is not applicable to device code (e.g., removal of unused standard wording such as garbage collection support, or host-defined behavior).

Table 38: C++23 Language Features Supported by NVCC for device code

Language Feature	C++23 Proposal	NVCC/CUDA Toolkit
Proposed resolution for core issues 411, 1656, and 2333; numeric and universal character escapes in character and string literals	P2029R4	✓
Literal Suffix for (signed) size_t	P0330R8	✓
Make () more optional for lambdas (Down with (!))	P1102R2	✓
if consteval	P1938R3	✓
Removing Garbage Collection Support	P2186R2	N/A
DR: C++ Identifier Syntax using Unicode Standard Annex 31	P1949R7	✓
DR: Allow Duplicate Attributes	P2156R1	✓
Narrowing contextual conversions to bool	P1401R5	×
Trimming whitespaces before line splicing	P2223R2	✓
Make declaration order layout mandated	P1847R4	✓
Mixed string literal concatenation	P2201R1	N/A
Non-literal variables (and labels and gotos) in const-expr functions	P2242R3	✓
Deducing this	P0847R7	✓
Consistent character literal encoding	P2316R2	✓
Add support for preprocessing directives elifdef and elifndef	P2334R1	✓
Character encoding of diagnostic text	P2246R1	✓
Extend init-statement to allow alias-declaration	P2360R0	✓
Change scope of lambda trailing-return-type	P2036R3	✓
Multidimensional subscript operator	P2128R6	✓
Character sets and encodings	P2314R4	✓
auto(x) and auto {x}	P0849R8	✓

continues on next page

Table 38 – continued from previous page

Language Feature	C++23 Proposal	NVCC/CUDA Toolkit
Missing feature test macros for C++20 core papers	P2493R0	✓
Attributes on lambda expression	P2173R1	✓
Support for #warning	P2437R1	✓
Remove non-encodable wide character literals and multicharacter wide character literals	P2362R3	✓
Labels at the end of compound statements (C compatibility)	P2324R2	✓
Delimited escape sequences	P2290R3	✓
Relaxing some constexpr restrictions	P2448R2	×
Simpler implicit move	P2266R3	✓
Named universal character escapes	P2071R2	✓
static operator()	P1169R4	✓
static operator[]	P2589R1	✓
Extended floating-point types and standard names	P1467R9	✓
Portable assumptions [[assume]]	P1774R8	✓
Support for UTF-8 as a portable source file encoding	P2295R6	✓
DR: char8_t Compatibility and Portability Fix	P2513R4	✓
DR: De-deprecating volatile bitwise compound assignment operations	P2327R1	×
DR: Relax requirements on wchar_t to match existing practices	P2460R2	N/A
DR: Using unknown pointers and references in constant expressions	P2280R4	×
DR: The Equality Operator You Are Looking For	P2468R2	×
Permitting static constexpr variables in constexpr functions	P2647R1	✓
Extending the lifetime of temporaries in range-based for loop initializer	P2644R1 P2718R0	✓
DR: consteval needs to propagate up	P2564R3	✓

5.3.6. CUDA C++ Standard Library

CUDA provides an implementation of the C++ Standard Library (STL), called `libc++`. The library presents the following benefits:

- The functionalities are available on both host and device.
- Compatible with all [Linux](#) and [Windows](#) platforms supported by the CUDA Toolkit.

- Compatible with all GPU architectures supported by the last two major versions of the CUDA Toolkit.
- Compatible with all CUDA Toolkits with the current and previous major versions.
- Provides C++17 backports of C++ Standard Library features available in recent standard versions, including C++20, C++23, and C++26.
- Supports extended data types, such as 128-bit integers (`__int128`), half-precision floats (`__half`), Bfloat16 (`__nv_bfloat16`), and quad-precision floats (`__float128`).
- Highly optimized for device code.

In addition, `libc++` provides **extended features** that are not available in the C++ Standard Library to improve productivity and application performance. Such features include mathematical functions, memory operations, synchronization primitives, container extensions, high-level abstractions of CUDA intrinsics, C++ PTX wrappers, and more.

`libc++` is available as part of the **CUDA Toolkit**, as well as part of the open-source **CCCL** repository.

5.3.7. C Standard Library Functions

5.3.7.1 `clock()` and `clock64()`

```
__host__ __device__ clock_t    clock();
__device__              long long clock64();
```

When executed in device code, it returns the value of a per-multiprocessor counter that increments every clock cycle. Sampling this counter at the beginning and end of a kernel, subtracting the two values, and recording the result for each thread provides an estimate of the number of clock cycles the device spends executing the thread. However, this value does not represent the actual number of clock cycles the device spends executing the thread's instructions. The former number is greater than the latter because threads are time-sliced.

Hint

- The corresponding **CUDA C++ function** `cuda::std::clock()` is provided in the `<cuda/std/ctime>` header.
- A portable C++ `<chrono>` implementation is also provided in the `<cuda/std/chrono>` header for similar purposes.

5.3.7.2 `printf()`

```
__host__ __device__ __tile__ int printf(const char* format[, arg, ...]);
```

The function prints formatted output from a kernel to a host-side output stream.

The in-kernel `printf()` function behaves similarly to the standard C library `printf()` function. Users should refer to their host system's manual pages for complete descriptions of `printf()` behavior. Essentially, the string passed in as `format` is output to a stream on the host.

The `printf()` command is executed like any other device-side function: per thread and in the context of the calling thread. In a multi-threaded kernel, a straightforward call to `printf()` will be executed by every thread using the data specified by that thread. Consequently, multiple versions of the output string will appear at the host stream, each corresponding to a thread that encountered the `printf()`.

Unlike the C standard `printf()`, which returns the number of characters printed, CUDA's `printf()` returns the number of arguments parsed. If no arguments follow the format string, 0 is returned. If the format string is NULL, -1 is returned. If an internal error occurs, -2 is returned.

Internally, `printf()` uses a shared data structure, so it is possible that calling `printf()` may alter the execution order of threads. In particular, a thread that calls `printf()` might take a longer execution path than a thread that does not call `printf()`, and the length of that path depends on the parameters of `printf()`. However, note that CUDA makes no guarantees about the order of thread execution except at explicit `__syncthreads()` barriers. Therefore, it is impossible to tell whether the order of execution has been modified by `printf()` or by other scheduling behaviors in the hardware.

The `printf()` function behaves differently in tile code than in SIMT device code. In tile code,

- ▶ The arguments may be tiles in addition to scalars. When passing a tile argument, each element of the tile is printed according to the corresponding format specifier.
- ▶ The return value is always the number of arguments provided even if the format string is NULL or an internal error occurs.
- ▶ The format string must be a literal.
- ▶ An error is issued if the number of arguments does not match the number of format specifiers. In SIMT device code, this scenario results in a warning.

Format Specifiers

As for standard `printf()`, format specifiers take the form: `%[flags][width][.precision][size]type`

The following fields are supported. See the widely available documentation for a complete description of all behaviors.

- ▶ Flags: #, ' ', 0, +, -
- ▶ Width: *, 0-9
- ▶ Precision: 0-9
- ▶ Size: h, l, ll
- ▶ Type: %cdiouxXpeEfgGaAs

Limitations

The final formatting of the `printf()` output takes place on the host system. This means that the format string must be understood by the compiler and C library of the host system. While every effort has been made to ensure that the format specifiers supported by CUDA's `printf()` function are a universal subset of those supported by the most common host compilers, the exact behavior will be dependent on the host operating system.

`printf()` accepts all valid combinations of flags and types. This is because it cannot determine what will and will not be valid on the host system where the final output is formatted. Consequently, output may be undefined if the program emits a format string containing invalid combinations.

The `printf()` function can accept up to 32 arguments, in addition to the format string. Any additional arguments will be ignored, and the format specifier will be output as is.

Due to the different sizes of the `long` type on Windows platforms (32-bit) and Linux platforms (64-bit), a kernel compiled on a Linux machine and then run on a Windows machine will produce corrupted

output for all format strings that include %ld. To ensure safety, it is recommended that the compilation and execution platforms match.

Host-Side Buffer

The output buffer for `printf()` is set to a fixed size before kernel launch. The buffer is circular, so if more output is produced during kernel execution than can fit in the buffer, older output is overwritten. The buffer is flushed only when one of the following actions is performed:

- ▶ Kernel launch via `<<< >>>` or `cuLaunchKernel()`: at the start of the launch, and if the `CUDA_LAUNCH_BLOCKING` environment variable is set to 1, at the end of the launch as well,
- ▶ Synchronization via `cudaDeviceSynchronize()`, `cuCtxSynchronize()`, `cudaStreamSynchronize()`, `cuStreamSynchronize()`, `cudaEventSynchronize()`, or `cuEventSynchronize()`,
- ▶ Memory copies via any blocking version of `cudaMemcpy*()` or `cuMemcpy*()`,
- ▶ Module loading/unloading via `cuModuleLoad()` or `cuModuleUnload()`,
- ▶ Context destruction via `cudaDeviceReset()` or `cuCtxDestroy()`.
- ▶ Prior to executing a stream callback added by `cudaLaunchHostFunc()` or `cuLaunchHostFunc()`.

Note that the buffer is not automatically flushed when the program exits.

The following API functions set and retrieve the size of the buffer used to transfer `printf()` arguments and internal metadata to the host. The default size is one megabyte.

- ▶ `cudaDeviceGetLimit(size_t* size, cudaLimitPrintfFifoSize)`
- ▶ `cudaDeviceSetLimit(cudaLimitPrintfFifoSize, size_t size)`

Examples

The following code sample:

```
#include <stdio.h>

__global__ void helloCUDA(float value) {
    printf("Hello thread %d, value=%f\n", threadIdx.x, value);
}

int main() {
    helloCUDA<<<1, 5>>>>(1.2345f);
    cudaDeviceSynchronize();
    return 0;
}
```

will output:

```
Hello thread 2, value=1.2345
Hello thread 1, value=1.2345
Hello thread 4, value=1.2345
Hello thread 0, value=1.2345
Hello thread 3, value=1.2345
```

Notice that each thread encounters the `printf()` command. Therefore, there are as many lines of output as there are threads in the grid.

See the example on [Compiler Explorer](#).

The following code sample:

```
#include <stdio.h>

__global__ void helloCUDA(float value) {
    if (threadIdx.x == 0)
        printf("Hello thread %d, value=%f\n", threadIdx.x, value);
}

int main() {
    helloCUDA<<<1, 5>>>>(1.2345f);
    cudaDeviceSynchronize();
    return 0;
}
```

will output:

```
Hello thread 0, value=1.2345
```

Clearly, the `if()` statement limits which threads call `printf()`, so only one line of output is seen.

See the example on [Compiler Explorer](#).

The following code sample:

```
#include "cuda_tile.h"
#include <cstdio>

namespace ct = cuda::tiles;

__tile_global__ void kernel() {
    auto ints = ct::iota<ct::tile<int, ct::shape<4, 4>>>>();
    printf("%i\n", ints);
}

int main() {
    kernel<<<1, 1>>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

will output:

```
[[0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11], [12, 13, 14, 15]]
```

5.3.7.3 memcpy() and memset()

```
__host__ __device__ __tile__ void* memcpy(void* dest, const void* src, size_t
↪size);
```

The function copies size bytes from the memory location pointed by src to the memory location pointed by dest.

```
__host__ __device__ __tile__ void* memset(void* ptr, int value, size_t size);
```

The function sets size bytes of memory block pointed by ptr to value, interpreted as an unsigned char.

Hint

It is suggested to use the `cuda::std::memcpy()` and `cuda::std::memset()` functions provided in the `<cuda/std/cstring>` header as safer versions of `memcpy` and `memset`.

5.3.7.4 malloc() and free()

```
__host__ __device__ void* malloc(size_t size);
// or cuda::std::malloc(), cuda::std::calloc() in the <cuda/std/cstdlib> header
```

The functions `malloc()` (device-side), `cuda::std::malloc()`, and `cuda::std::calloc()` allocate at least size bytes from the device heap and return a pointer to the allocated memory. If insufficient memory exists to fulfill the request, it returns NULL. The returned pointer is guaranteed to be aligned to a 16-byte boundary.

```
__device__ void* __nv_aligned_device_malloc(size_t size, size_t align);
// or cuda::std::aligned_alloc() in the <cuda/std/cstdlib> header
```

The functions `__nv_aligned_device_malloc()` and C++ `cuda::std::aligned_alloc()` allocate at least size bytes from the device heap and return a pointer to the allocated memory. If there is insufficient memory to fulfill the requested size or alignment, it returns NULL. The address of the allocated memory is a multiple of align. align must be a non-zero power of two.

```
__host__ __device__ void free(void* ptr);
// or cuda::std::free() in the <cuda/std/cstdlib> header
```

The device-side functions `free()` and `cuda::std::free()` deallocate the memory pointed to by ptr, which must have been returned by a previous call to `malloc()`, `cuda::std::malloc()`, `cuda::std::calloc()`, `__nv_aligned_device_malloc()`, or `cuda::std::aligned_alloc()`. If ptr is NULL, the call to `free()` or `cuda::std::free()` is ignored. Repeated calls to `free()` or `cuda::std::free()` with the same ptr have undefined behavior.

Memory allocated by a given CUDA thread via `malloc()`, `cuda::std::malloc()`, `cuda::std::calloc()`, `__nv_aligned_device_malloc()`, or `cuda::std::aligned_alloc()` remain allocated for the lifetime of the CUDA context, or until it is explicitly released by a call to `free()` or `cuda::std::free()`. This memory can be used by other CUDA threads, even those from subsequent kernel launches. Any CUDA thread can free memory allocated by another thread; however, care should be taken to ensure that the same pointer is not freed more than once.

Heap Memory API

The size of the device memory heap must be specified before any program that allocates or frees memory in device code, including the `new` and `delete` keywords. If any program uses the device memory heap without explicitly specifying the heap size, a default heap of eight megabytes is allocated.

The following API functions get and set the heap size:

- ▶ `cudaDeviceGetLimit(size_t* size, cudaLimitMallocHeapSize)`
- ▶ `cudaDeviceSetLimit(cudaLimitMallocHeapSize, size_t size)`

The heap size granted will be at least `size` bytes. `cuCtxGetLimit()` and `cudaDeviceGetLimit()` return the currently requested heap size.

The actual memory allocation for the heap occurs when a module is loaded into the context, either explicitly through the CUDA driver API (see [Module](#)) or implicitly through the CUDA runtime API. If memory allocation fails, the module load generates a `CUDA_ERROR_SHARED_OBJECT_INIT_FAILED` error.

The heap size cannot be changed after a module has been loaded, and it does not dynamically resize according to need.

The memory reserved for the device heap is in addition to the memory allocated through host-side CUDA API calls such as `cudaMalloc()`.

Interoperability with the Host Memory API

Memory allocated via the device-side functions `malloc()`, `cuda::std::malloc()`, `cuda::std::calloc()`, `__nv_aligned_device_malloc()`, `cuda::std::aligned_alloc()`, or the `new` keyword cannot be used or freed with runtime or driver API calls such as `cudaMalloc`, `cudaMemcpy`, or `cudaMemset`. Similarly, memory allocated via the host runtime API cannot be freed using the device-side functions `free()`, `cuda::std::free()`, or the `delete` keyword.

Per-Thread Allocation example:

```
#include <stdlib.h>
#include <stdio.h>

__global__ void single_thread_allocation_kernel() {
    size_t size = 123;
    char* ptr = (char*) malloc(size);
    memset(ptr, 0, size);
    printf("Thread %d got pointer: %p\n", threadIdx.x, ptr);
    free(ptr);
}

int main() {
    // Set a heap size of 128 megabytes.
    // Note that this must be done before any kernel is launched.
    cudaDeviceSetLimit(cudaLimitMallocHeapSize, 128 * 1024 * 1024);
    single_thread_allocation_kernel<<<1, 5>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

will output:

```
Thread 0 got pointer: 0x20d5ffe20
Thread 1 got pointer: 0x20d5ffec0
Thread 2 got pointer: 0x20d5fff60
Thread 3 got pointer: 0x20d5f97c0
Thread 4 got pointer: 0x20d5f9720
```

Notice how each thread encounters the `malloc()` and `memset()` commands and so receives and initializes its own allocation.

See the example on [Compiler Explorer](#).

Per-Thread-Block Allocation example:

```
#include <stdlib.h>

__global__ void block_level_allocation_kernel() {
    __shared__ int* data;
    // The first thread in the block performs the allocation and shares the
    // pointer
    // with all other threads through shared memory, so that access can be
    // coalesced.
    if (threadIdx.x == 0) {
        size_t size = blockDim.x * 64; // 64 bytes per thread are allocated.
        data = (int*) malloc(size);
    }
    __syncthreads();
    // Check for failure
    if (data == nullptr)
        return;

    // Threads index into the memory, ensuring coalescence
    for (int i = 0; i < 64; ++i)
        data[i * blockDim.x + threadIdx.x] = threadIdx.x;
    // Ensure all threads complete before freeing
    __syncthreads();

    // Only one thread may free the memory!
    if (threadIdx.x == 0)
        free(data);
}

int main() {
    cudaDeviceSetLimit(cudaLimitMallocHeapSize, 128 * 1024 * 1024);
    block_level_allocation_kernel<<<10, 128>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

See the example on [Compiler Explorer](#).

Allocation Persisting Between Kernel Launches example:

```

#include <stdlib.h>
#include <stdio.h>

const int NUM_BLOCKS = 20;

__device__ int* data_ptrs[NUM_BLOCKS]; // Per-block pointer

__global__ void allocate_memory_kernel() {
    // Only the first thread in the block performs the allocation
    // since we need only one allocation per block.
    if (threadIdx.x == 0)
        data_ptrs[blockIdx.x] = (int*) malloc(blockDim.x * 4);
    __syncthreads();
    // Check for failure
    if (data_ptrs[blockIdx.x] == nullptr)
        return;
    // Zero the data with all threads in parallel
    data_ptrs[blockIdx.x][threadIdx.x] = 0;
}

// Simple example: store the thread ID into each element
__global__ void use_memory_kernel() {
    int* ptr = data_ptrs[blockIdx.x];
    if (ptr != nullptr)
        ptr[threadIdx.x] += threadIdx.x;
}

// Print the content of the buffer before freeing it
__global__ void free_memory_kernel() {
    int* ptr = data_ptrs[blockIdx.x];
    if (ptr != nullptr)
        printf("Block %d, Thread %d: final value = %d\n",
            blockIdx.x, threadIdx.x, ptr[threadIdx.x]);
    // Only free from one thread!
    if (threadIdx.x == 0)
        free(ptr);
}

int main() {
    cudaDeviceSetLimit(cudaLimitMallocHeapSize, 128*1024*1024);
    // Allocate memory
    allocate_memory_kernel<<<NUM_BLOCKS, 10>>>();

    // Use memory
    use_memory_kernel<<<NUM_BLOCKS, 10>>>();
    use_memory_kernel<<<NUM_BLOCKS, 10>>>();
    use_memory_kernel<<<NUM_BLOCKS, 10>>>();

    // Free memory
    free_memory_kernel<<<NUM_BLOCKS, 10>>>();
    cudaDeviceSynchronize();
    return 0;
}

```

See the example on [Compiler Explorer](#).

5.3.7.5 `alloca()`

```
__host__ __device__ void* alloca(size_t size);
```

The `alloca()` function allocates `size` bytes of memory within the caller's stack frame. The returned value is a pointer to the allocated memory. When the function is invoked from device code, the beginning of the memory is 16-byte aligned. The memory is automatically freed when the caller returns from `alloca()`.

Note

On the Windows platform, the `<malloc.h>` header file must be included before using the `alloca()` function. Calls to `alloca()` may cause the stack to overflow; the user needs to adjust the stack size accordingly.

Example:

```
__device__ void device_function(int num_items) {
    int4* ptr = (int4*) alloca(num_items * sizeof(int4));
    // use of ptr
    ...
}
```

5.3.8. Lambda Expressions

The compiler determines the execution space of a lambda expression or closure type (C++11) by associating it with the execution space of the innermost enclosing function scope. If there is no enclosing function scope, the execution space is specified as `__host__`.

The execution space can also be specified explicitly with the *extended lambda syntax*.

Examples:

```
auto global_lambda = [](){ return 0; }; // __host__

void host_function() {
    auto lambda1 = [](){ return 1; }; // __host__
    [](){ return 3; }; // __host__, closure type (body of a
    ↪ lambda expression)
}

__device__ void device_function() {
    auto lambda2 = [](){ return 2; }; // __device__
}

__global__ void kernel_function(void) {
    auto lambda3 = [](){ return 3; }; // __device__
}
```

(continues on next page)

(continued from previous page)

```

__host__ __device__ void host_device_function() {
    auto lambda4 = [](){ return 4; }; // __host__ __device__
}

__tile__ void tile_function() {
    auto lambda5 = [](){ return 5; }; // __tile__
}

__tile__ __device__ void tile_device_function() {
    auto lambda6 = [](){ return 6; }; // __tile__ __device__
}

using function_ptr_t = int (*)(*);

__device__ void device_function(float value,
                                function_ptr_t ptr = [](){ return 4; } /* __
→host__ */) {}

```

See the example on [Compiler Explorer](#).

5.3.8.1 Lambda Expressions and __global__ Function Parameters

A lambda expression or a closure type can only be used as an argument to a __global__ function if its execution space is __device__ or __host__ __device__. Global or namespace scope lambda expressions cannot be used as arguments in a __global__ function.

Examples:

```

template <typename T>
__global__ void kernel(T input) {}

__device__ void device_function() {
    // device kernel call requires separate compilation (-rdc=true flag)
    kernel<<<1, 1>>>([](){});
    kernel<<<1, 1>>>([] __device__() {}); // extended lambda
    kernel<<<1, 1>>>([] __host__ __device__() {}); // extended lambda
}

auto global_lambda = [] __host__ __device__() {};

void host_function() {
    kernel<<<1, 1>>>([] __device__() {}); // CORRECT, extended
→lambda
    kernel<<<1, 1>>>([] __host__ __device__() {}); // CORRECT, extended
→lambda
    // kernel<<<1, 1>>>([](){}); // ERROR, closure type
→with host execution space
    // kernel<<<1, 1>>>(global_lambda); // ERROR, extended lambda,
→but at global scope
}

```

See the example on [Compiler Explorer](#).

5.3.8.2 Extended Lambdas

The `nvcc` flag `--extended-lambda` allows explicit annotations of execution spaces in a lambda expression. These annotations should appear after the lambda introducer and before the optional lambda declarator.

`nvcc` defines the macro `__CUDACC_EXTENDED_LAMBDA__` when the `--extended-lambda` flag is specified.

- An *extended lambda* is defined within the scope of an immediate or nested block of a `__host__` or `__host__ __device__` function.
- An *extended device lambda* is a lambda expression annotated with the `__device__` keyword.
- An *extended host-device lambda* is a lambda expression annotated with the `__host__ __device__` keywords.

Unlike standard lambda expressions, extended lambdas can be used as type arguments in `__global__` functions.

Example:

```
void host_function() {
    auto lambda1 = [] {}; // NOT an extended lambda: no
    ↪explicit execution space annotations
    auto lambda2 = [] __device__ {}; // extended lambda
    auto lambda3 = [] __host__ __device__ {}; // extended lambda
    auto lambda4 = [] __host__ {}; // NOT an extended lambda
}

__host__ __device__ void host_device_function() {
    auto lambda1 = [] {}; // NOT an extended lambda: no
    ↪explicit execution space annotations
    auto lambda2 = [] __device__ {}; // extended lambda
    auto lambda3 = [] __host__ __device__ {}; // extended lambda
    auto lambda4 = [] __host__ {}; // NOT an extended lambda
}

__device__ void device_function() {
    // none of the lambdas within this function are extended lambdas,
    // because the enclosing function is not a __host__ or __host__ __device__
    ↪function.
    auto lambda1 = [] {};
    auto lambda2 = [] __device__ {};
    auto lambda3 = [] __host__ __device__ {};
    auto lambda4 = [] __host__ {};
}

auto global_lambda = [] __host__ __device__ { }; // NOT an extended lambda
    ↪because it is not defined
    // within a __host__ or __
    ↪host__ __device__ function
```

5.3.8.3 Extended Lambda Type Traits

The compiler provides type traits to detect closure types for extended lambdas at compile time.

```
bool __nv_is_extended_device_lambda_closure_type(type);
```

The function returns `true` if `type` is the closure class created for an extended `__device__` lambda, `false` otherwise.

```
bool __nv_is_extended_device_lambda_with_preserved_return_type(type);
```

The function returns `true` if `type` is the closure class created for an extended `__device__` lambda and the lambda is defined with trailing return type, `false` otherwise. If the trailing return type definition refers to any lambda parameter name, the return type is not preserved.

```
bool __nv_is_extended_host_device_lambda_closure_type(type);
```

The function returns `true` if `type` is the closure class created for an extended `__host__ __device__` lambda, `false` otherwise.

The lambda type traits can be used in all compilation modes, regardless of whether lambdas or extended lambdas are enabled. The traits will always return `false` if extended lambda mode is inactive.

Example:

```
auto lambda0 = [] __host__ __device__ { };

void host_function() {
    auto lambda1 = [] { };
    auto lambda2 = [] __device__ { };
    auto lambda3 = [] __host__ __device__ { };
    auto lambda4 = [] __device__ () -> double { return 3.14; };
    auto lambda5 = [] __device__ (int x) -> decltype(&x) { return 0; };

    using lambda0_t = decltype(lambda0);
    using lambda1_t = decltype(lambda1);
    using lambda2_t = decltype(lambda2);
    using lambda3_t = decltype(lambda3);
    using lambda4_t = decltype(lambda4);
    using lambda5_t = decltype(lambda5);

    // 'lambda0' is not an extended lambda because it is defined outside
    // function scope
    static_assert(!__nv_is_extended_device_lambda_closure_type(lambda0_t));
    static_assert(!__nv_is_extended_device_lambda_with_preserved_return_
    type(lambda0_t));
    static_assert(!__nv_is_extended_host_device_lambda_closure_type(lambda0_
    t));

    // 'lambda1' is not an extended lambda because it has no execution space
    // annotations
    static_assert(!__nv_is_extended_device_lambda_closure_type(lambda1_t));
    static_assert(!__nv_is_extended_device_lambda_with_preserved_return_
    type(lambda1_t));
```

(continues on next page)

(continued from previous page)

```

    static_assert(!__nv_is_extended_host_device_lambda_closure_type(lambda1_
→t));

    // 'lambda2' is an extended device-only lambda
    static_assert(__nv_is_extended_device_lambda_closure_type(lambda2_t));
    static_assert(!__nv_is_extended_device_lambda_with_preserved_return_
→type(lambda2_t));
    static_assert(!__nv_is_extended_host_device_lambda_closure_type(lambda2_
→t));

    // 'lambda3' is an extended host-device lambda
    static_assert(!__nv_is_extended_device_lambda_closure_type(lambda3_t));
    static_assert(!__nv_is_extended_device_lambda_with_preserved_return_
→type(lambda3_t));
    static_assert(__nv_is_extended_host_device_lambda_closure_type(lambda3_
→t));

    // 'lambda4' is an extended device-only lambda with preserved return type
    static_assert(__nv_is_extended_device_lambda_closure_type(lambda4_t));
    static_assert(__nv_is_extended_device_lambda_with_preserved_return_
→type(lambda4_t));
    static_assert(!__nv_is_extended_host_device_lambda_closure_type(lambda4_
→t));

    // 'lambda5' is not an extended device-only lambda with preserved return
→type
    // because it references the operator()'s parameter types in the trailing
→return type.
    static_assert(__nv_is_extended_device_lambda_closure_type(lambda5_t));
    static_assert(!__nv_is_extended_device_lambda_with_preserved_return_
→type(lambda5_t));
    static_assert(!__nv_is_extended_host_device_lambda_closure_type(lambda5_
→t));
}

```

5.3.8.4 Extended Lambda Restrictions

Before invoking the host compiler, the CUDA compiler replaces an extended lambda expression with an instance of a placeholder type defined in namespace scope. The placeholder type's template argument requires taking the address of a function that encloses the original extended lambda expression. This is necessary for correctly executing any `__global__` function template whose template argument involves the closure type of an extended lambda. The enclosing function is computed as follows.

By definition, an extended lambda is present within the immediate or nested block scope of a `__host__` or `__host__ __device__` function.

- If the function is not the `operator()` of a lambda expression, it is considered the enclosing function for the extended lambda.
- Otherwise, the extended lambda is defined within the immediate or nested block scope of the `operator()` of one or more enclosing lambda expressions.
 - If the outermost lambda expression is defined within the immediate or nested block scope of a function `F`, then `F` is the computed enclosing function.

- Otherwise, the enclosing function does not exist.

Example:

```
void host_function() {
    auto lambda1 = [] __device__ { }; // enclosing function for lambda1 is
    → "host_function()"
    auto lambda2 = [] {
        auto lambda3 = [] {
            auto lambda4 = [] __host__ __device__ { }; // enclosing function
            → for lambda4 is "host_function"
        };
    };
}

auto global_lambda = [] {
    auto lambda5 = [] __host__ __device__ { }; // enclosing function for
    → lambda5 does not exist
};
```

Extended Lambda Restrictions

1. An extended lambda cannot be defined inside another extended lambda expression. Example:

```
void host_function() {
    auto lambda1 = [] __host__ __device__ {
        // ERROR, extended lambda defined within another extended lambda
        auto lambda2 = [] __host__ __device__ { };
    };
}
```

2. An extended lambda cannot be defined inside a generic lambda expression. Example:

```
void host_function() {
    auto lambda1 = [] (auto) {
        // ERROR, extended lambda defined within a generic lambda
        auto lambda2 = [] __host__ __device__ { };
    };
}
```

3. If an extended lambda is defined within the immediate or nested block scope of one or more nested lambda expressions, then the outermost lambda expression must be defined within the immediate or nested block scope of a function. Example:

```
auto lambda1 = [] {
    // ERROR, outer enclosing lambda is not defined within a non-lambda-
    → operator() function
    auto lambda2 = [] __host__ __device__ { };
};
```

4. The enclosing function of the extended lambda must be named, and its address must be accessible. If the enclosing function is a class member, the following conditions must be met:
 - All classes enclosing the member function must have a name.

- The member function must not have private or protected access within its parent class.
- All enclosing classes must not have private or protected access within their respective parent classes.

Example:

```
void host_function() {
    auto lambda1 = [] __device__ { return 0; }; // OK
    {
        auto lambda2 = [] __device__ { return 0; }; // OK
        auto lambda3 = [] __device__ __host__ { return 0; }; // OK
    }
}

struct MyStruct1 {
    MyStruct1() {
        auto lambda4 = [] __device__ { return 0; }; // ERROR, address of
        →the enclosing function is not accessible
    }
};

class MyStruct2 {
    void foo() {
        auto temp1 = [] __device__ { return 10; }; // ERROR, enclosing
        →function has private access in parent class
    }

    struct MyStruct3 {
        void foo() {
            auto temp1 = [] __device__ { return 10; }; // ERROR,
            →enclosing class MyStruct3 has private access in its parent class
        }
    };
};
```

5. At the point where the extended lambda has been defined, it must be possible to unambiguously take the address of the enclosing routine. However, this may not always be feasible, for example, when an alias declaration shadows a template type argument with the same name. Example:

```
template <typename T>
struct A {
    using Bar = void;
    void test();
};

template<>
struct A<void> { };

template <typename Bar>
void A<Bar>::test() {
    // In code sent to host compiler, nvcc will inject an address
    →expression here, of the form:
    // (void (A< Bar> ::*)(void))(&A::test))
    // However, the class typedef 'Bar' (to void) shadows the template
```

(continues on next page)

(continued from previous page)

```

→argument 'Bar',
    // causing the address expression in A<int>::test to actually refer
→to:
    // (void (A< void> ::*)(void))(&A::test))
    // which doesn't take the address of the enclosing routine 'A<int>
→::test' correctly.
    auto lambda1 = [] __host__ __device__ { return 4; };
}

int main() {
    A<int> var;
    var.test();
}

```

6. An extended lambda cannot be defined in a class that is local to a function. Example:

```

void host_function() {
    struct MyStruct {
        void bar() {
            // ERROR, bar() is member of a class that is local to a
→function
            auto lambda2 = [] __host__ __device__ { return 0; };
        }
    };
}

```

7. The enclosing function for an extended lambda cannot have deduced return type. Example:

```

auto host_function() {
    // ERROR, the return type of host_function() is deduced
    auto lambda3 = [] __host__ __device__ { return 0; };
}

```

8. A host-device extended lambda cannot be a generic lambda, namely a lambda with an auto parameter type. Example:

```

void host_function() {
    // ERROR, __host__ __device__ extended lambdas cannot be a generic
→lambda
    auto lambda1 = [] __host__ __device__ (auto i) { return i; };

    // ERROR, a host-device extended lambda cannot be a generic lambda
    auto lambda2 = [] __host__ __device__ (auto... i) {
        return sizeof...(i);
    };
}

```

9. If the enclosing function is an instantiation of a function or member template, or if the function is a member of a class template, then the template(s) must satisfy the following constraints:

- ▶ The template must have at most one variadic parameter, and it must be listed last in the template parameter list.
- ▶ The template parameters must be named.

- The template instantiation argument types cannot involve types that are either local to a function (except for closure types for extended lambdas), or are private or protected class members.

Example 1:

```
template <template <typename...> class T,
          typename... P1,
          typename... P2>
void bar1(const T<P1...>, const T<P2...>) {
    // ERROR, enclosing function has multiple parameter packs
    auto lambda = [] __device__ { return 10; };
}

template <template <typename...> class T,
          typename... P1,
          typename    T2>
void bar2(const T<P1...>, T2) {
    // ERROR, for enclosing function, the parameter pack is not last in
    // the template parameter list
    auto lambda = [] __device__ { return 10; };
}

template <typename T, T>
void bar3() {
    // ERROR, for enclosing function, the second template parameter is not
    // named
    auto lambda = [] __device__ { return 10; };
}
```

Example 2:

```
template <typename T>
void bar4() {
    auto lambda1 = [] __device__ { return 10; };
}

class MyStruct {
    struct MyNestedStruct {};

    friend int main();
};

int main() {
    struct MyLocalStruct {};
    // ERROR, enclosing function for device lambda in bar4() is
    // instantiated with a type local to main
    bar4<MyLocalStruct>();

    // ERROR, enclosing function for device lambda in bar4 is instantiated
    // with a type
    // that is a private member of a class
    bar4<MyStruct::MyNestedStruct>();
}
```

10. With Microsoft Visual Studio host compilers, the enclosing function must have external linkage. This restriction exists because the host compiler does not support using the addresses of non-extern linkage functions as template arguments. The CUDA compiler transformations require these addresses to support extended lambdas.
11. With Microsoft Visual Studio host compilers, an extended lambda shall not be defined within the body of an `if constexpr` block.
12. An extended lambda has the following restrictions on captured variables:
 - ▶ The variable may be passed by value to a sequence of helper functions in the code sent to the host compiler before being used to directly initialize the field of the class type representing the closure type for the extended lambda. However, the C++ standard specifies that the captured variable should be used for direct initialization of the closure type's field.
 - ▶ A variable can only be captured by value.
 - ▶ A variable of array type cannot be captured if the number of array dimensions is greater than 7.
 - ▶ For an array-type variable, the array field of the closure type is first default-initialized and then each array element is copy-assigned from the corresponding element of the captured array variable in the code sent to the host compiler. Therefore, the array element type must be both default-constructible and copy-assignable in the host code.
 - ▶ A function parameter that is an element of a variadic argument pack cannot be captured.
 - ▶ The captured variable type cannot be local to a function, except for extended lambda closure types, or `private` or `protected` class members.
 - ▶ Init-capture is not supported for host-device extended lambdas. However, it is supported for device extended lambdas, except when the initializer is an array or of type `std::initializer_list`.
 - ▶ The function call operator for an extended lambda is not a `constexpr`. The closure type of an extended lambda is not a literal type. The `constexpr` and `constexpr` specifiers cannot be used when declaring an extended lambda.
 - ▶ A variable cannot be implicitly captured inside an `if-constexpr` block that is lexically nested inside an extended lambda unless the variable has been implicitly captured outside the `if-constexpr` block or appears in the extended lambda's explicit capture list.

Examples:

```
void host_function() {
    // CORRECT, an init-capture is allowed for an extended device-only
    →lambda
    auto lambda1 = [x = 1] __device__ () { return x; };

    // ERROR, an init-capture is not allowed for an extended host-device
    →lambda
    auto lambda2 = [x = 1] __host__ __device__ () { return x; };

    int a = 1;
    // ERROR, an extended __device__ lambda cannot capture variables by
    →reference
    auto lambda3 = [&a] __device__ () { return a; };

    // ERROR, by-reference capture is not allowed for an extended device-
```

(continues on next page)

(continued from previous page)

```

→only lambda
    auto lambda4 = [&x = a] __device__ () { return x; };

    struct MyStruct {};
    MyStruct s1;
    // ERROR, a type local to a function cannot be used in the type of a
→captured variable
    auto lambda6 = [s1] __device__ () { };

    // ERROR, an init-capture cannot be of type std::initializer_list
    auto lambda7 = [x = {11}] __device__ () { };

    std::initializer_list<int> b = {11,22,33};
    // ERROR, an init-capture cannot be of type std::initializer_list
    auto lambda8 = [x = b] __device__ () { };

    int var = 4;
    auto lambda9 = [=] __device__ {
        int result = 0;
        if constexpr(false) {
            //ERROR, An extended device-only lambda cannot first-capture
→'var' in if-constexpr context
            result += var;
        }
        return result;
    };

    auto lambda10 = [var] __device__ {
        int result = 0;
        if constexpr(false) {
            // CORRECT, 'var' already listed in explicit capture list for
→the extended lambda
            result += var;
        }
        return result;
    };

    auto lambda11 = [=] __device__ {
        int result = var;
        if constexpr(false) {
            // CORRECT, 'var' already implicit captured outside the 'if-
→constexpr' block
            result += var;
        }
        return result;
    };
}

```

13. When parsing a function, the CUDA compiler assigns a counter value to each extended lambda in the function. This counter value is used in the substituted named type that is passed to the host compiler. Therefore, the presence or absence of an extended lambda within a function should not depend on a particular value of `__CUDA_ARCH__`, nor on `__CUDA_ARCH__` being undefined. Example:

```

template <typename T>
__global__ void kernel(T in) { in(); }

__host__ __device__ void host_device_function() {
    // ERROR, the number and relative declaration order of
    // extended lambdas depend on __CUDA_ARCH__
#ifdef __CUDA_ARCH__
    auto lambda1 = [] __device__ { return 0; };
    auto lambda2 = [] __host__ __device__ { return 10; };
#elseif
    auto lambda3 = [] __device__ { return 4; };
    kernel<<<1, 1>>>(lambda3);
}

```

14. As described above, the CUDA compiler replaces a device extended lambda defined in a host function with a placeholder type defined in namespace scope. The placeholder type does not define an `operator()` function equivalent to the original lambda declaration unless the trait `__nv_is_extended_device_lambda_with_preserved_return_type()` returns true for the closure type of the extended lambda. Therefore, an attempt to determine the return type or parameter types of the `operator()` function of such a lambda may work incorrectly in host code because the code processed by the host compiler is semantically different from the input code processed by the CUDA compiler. However, introspecting the return type or parameter types of the `operator()` function within device code is acceptable. Note that this restriction does not apply to host or device extended lambdas for which the trait `__nv_is_extended_device_lambda_with_preserved_return_type()` returns true. Example:

```

#include <cuda/std/type_traits>

const char& getRef(const char* p) { return *p; }

void foo() {
    auto lambda1 = [] __device__ { return "10"; };

    // ERROR, attempt to extract the return type of a device lambda in
    →host code
    cuda::std::result_of<decltype(lambda1())>::type xx1 = "abc";

    auto lambda2 = [] __host__ __device__ { return "10"; };

    // CORRECT, lambda2 represents a host-device extended lambda
    cuda::std::result_of<decltype(lambda2())>::type xx2 = "abc";

    auto lambda3 = [] __device__ () -> const char* { return "10"; };

    // CORRECT, lambda3 represents a device extended lambda with preserved
    →return type
    cuda::std::result_of<decltype(lambda3())>::type xx2 = "abc";
    static_assert(cuda::std::is_same_v<cuda::std::result_of
    →<decltype(lambda3())>::type, const char*>);

    auto lambda4 = [] __device__ (char x) -> decltype(getRef(&x)) {
    →return 0; };

```

(continues on next page)

(continued from previous page)

```

    // lambda4's return type is not preserved because it references the
    ↪operator()'s
    // parameter types in the trailing return type.
    static_assert(!__nv_is_extended_device_lambda_with_preserved_return_
    ↪type(decltype(lambda4)));
}

```

15. For an extended device-only lambda:

- ▶ Introspection of the parameter type of `operator()` is only supported in device code.
- ▶ Introspection of the return type of `operator()` is supported only in device code, unless the trait function `__nv_is_extended_device_lambda_with_preserved_return_type()` returns true.

16. If an extended lambda is passed from host to device code as an argument to a `__global__` function, for example, then any expression in the lambda's body that captures variables must remain unchanged, regardless of whether the `__CUDA_ARCH__` macro is defined and what value it has. This restriction arises because the lambda's closure class layout depends on the order in which the compiler encounters the captured variables when processing the lambda expression. The program may execute incorrectly if the closure class layout differs between device and host compilations. Example:

```

__device__ int result;

template <typename T>
__global__ void kernel(T in) { result = in(); }

void foo(void) {
    int x1 = 1;
    // ERROR, "x1" is only captured when __CUDA_ARCH__ is defined.
    auto lambda1 = [=] __host__ __device__ {
#ifdef __CUDA_ARCH__
        return x1 + 1;
    #else
        return 10;
    #endif
    };
    kernel<<<1, 1>>>(lambda1);
}

```

17. As previously described, the CUDA compiler replaces an extended device-only lambda expression with a placeholder type instance in the code sent to the host compiler. The placeholder type does not define a pointer-to-function conversion operator in the host code; however, the conversion operator is provided in the device code. Note that this restriction does not apply to host-device extended lambdas. Example:

```

template <typename T>
__global__ void kernel(T in) {
    int (*fp)(double) = in;
    fp(0); // CORRECT, conversion in device code is supported
    auto lambda1 = [] (double) { return 1; };
}

```

(continues on next page)

(continued from previous page)

```

void foo() {
    auto lambda_device = [] __device__ (double) { return 1; };
    auto lambda_host_device = [] __host__ __device__ (double) { return 1;
→};
    kernel<<<1, 1>>>(lambda_device);
    kernel<<<1, 1>>>(lambda_host_device);

    // CORRECT, conversion for a __host__ __device__ lambda is supported
→in host code
    int (*fp1)(double) = lambda_host_device;

    // ERROR, conversion for a device lambda is not supported in host code
    int (*fp2)(double) = lambda_device;
}

```

18. As previously described, the CUDA compiler replaces an extended device-only or host-device lambda expression with a placeholder type instance in the code sent to the host compiler. This placeholder type may define C++ special member functions, such as constructors and destructors. Consequently, some standard C++ type traits may yield different results for the closure type of the extended lambda in the CUDA front-end compiler than in the host compiler. The following type traits are affected: `std::is_trivially_copyable`, `std::is_trivially_constructible`, `std::is_trivially_copy_constructible`, `std::is_trivially_move_constructible`, `std::is_trivially_destructible`. Care must be taken to ensure that the results of these traits are not used in the instantiation of the `__global__`, `__device__`, `__constant__`, or `__managed__` function or variable templates. Example:

```

#include <cstdio>
#include <type_traits>

template <bool b>
void __global__ kernel() { printf("hi"); }

template <typename T>
void kernel_launch() {
    // ERROR, this kernel launch may fail, because CUDA frontend compiler
→and host compiler
    // may disagree on the result of std::is_trivially_copyable_v
→trait on the
    // closure type of the extended lambda
    kernel<std::is_trivially_copyable_v<T>><<<1,1>>>();
    cudaDeviceSynchronize();
}

int main() {
    int x = 0;
    auto lambda1 = [=] __host__ __device__ () { return x; };
    kernel_launch<decltype(lambda1)>();
}

```

The CUDA compiler will generate compiler diagnostics for a subset of cases described in 1–12; no diagnostic will be generated for cases 13–17, but the host compiler may fail to compile the generated code.

5.3.8.5 Host-Device Lambda Optimization Notes

Unlike device-only lambdas, host-device lambdas can be called from host code. As previously mentioned, the CUDA compiler replaces an extended lambda expression defined in host code with an instance of a named placeholder type. The placeholder type for an extended host-device lambda invokes the original lambda's `operator()` with an indirect function call. The traits will always return false if extended lambda mode is not active.

The presence of an indirect function call may cause the host compiler to optimize an extended host-device lambda less than lambdas that are implicitly or explicitly `__host__` only. In the latter case, the host compiler can easily inline the lambda body into the calling context. However, when it encounters an extended host-device lambda, the host compiler may not be able to easily inline the original lambda body.

5.3.8.6 `*this` Capture By-Value

According to C++11/C++14 rules, when a lambda is defined within a non-static class member function and the lambda's body refers to a class member variable, the `this` pointer of the class must be captured by value rather than the referenced member variable. If the lambda is an extended device-only or host-device lambda defined in a host function and executed on the GPU, accessing the referenced member variable on the GPU will cause a runtime error if the `this` pointer points to host memory.

Example:

```
#include <cstdio>

template <typename T>
__global__ void foo(T in) { printf("value = %d\n", in()); }

struct MyStruct {
    int var;

    __host__ __device__ MyStruct() : var(10) {};

    void run() {
        auto lambda1 = [=] __device__ {
            // reference to "var" causes the 'this' pointer (MyStruct*) to be
            ↪captured by value
            return var + 1;
        };
        // Kernel launch fails at run time because 'this->var' is not
        ↪accessible from the GPU
        foo<<<1, 1>>>>(lambda1);
        cudaDeviceSynchronize();
    }
};

int main() {
    MyStruct s1;
    s1.run();
}
```

C++17 solves this problem by introducing a new `*this` capture mode. In this mode, the compiler copies the object denoted by `*this` instead of capturing the `this` pointer by value. The `*this` capture mode is described in more detail in [P0018R3](#).

The CUDA compiler supports the `*this` capture mode for lambdas defined within `__device__` and `__global__` functions and for extended device-only lambdas defined in host code, when the `--extended-lambda` flag is used.

Here's the above example modified to use `*this` capture mode:

```
#include <stdio>

template <typename T>
__global__ void foo(T in) { printf("\n value = %d", in()); }

struct MyStruct {
    int var;
    __host__ __device__ MyStruct() : var(10) { };

    void run() {
        // note the "*this" capture specification
        auto lambda1 = [=, *this] __device__ {
            // reference to "var" causes the object denoted by '*this' to be
            // captured by
            // value, and the GPU code will access 'copy_of_star_this->var'
            return var + 1;
        };
        // Kernel launch succeeds
        foo<<<1, 1>>>>(lambda1);
        cudaDeviceSynchronize();
    }
};

int main() {
    MyStruct s1;
    s1.run();
}
```

`*this` capture mode is not allowed for non-annotated lambdas defined in host code, or for extended host-device lambdas, unless `*this` capture is enabled by the selected language dialect. The following are examples of supported and unsupported usage:

```
struct MyStruct {
    int var;
    __host__ __device__ MyStruct() : var(10) { };

    void host_function() {
        // CORRECT, use in an extended device-only lambda
        auto lambda1 = [=, *this] __device__ { return var; };

        // Use in an extended host-device lambda
        // Error if *this capture not enabled by language dialect
        auto lambda2 = [=, *this] __host__ __device__ { return var; };

        // Use in an non-annotated lambda in host function
        // Error if *this capture not enabled by language dialect
        auto lambda3 = [=, *this] { return var; };
    }
}
```

(continues on next page)

(continued from previous page)

```

__device__ void device_function() {
    // CORRECT, use in a lambda defined in a device-only function
    auto lambda1 = [=, *this] __device__ { return var; };

    // CORRECT, use in a lambda defined in a device-only function
    auto lambda2 = [=, *this] __host__ __device__ { return var; };

    // CORRECT, use in a lambda defined in a device-only function
    auto lambda3 = [=, *this] { return var; };
}

__host__ __device__ void host_device_function() {
    // CORRECT, use in an extended device-only lambda
    auto lambda1 = [=, *this] __device__ { return var; };

    // Use in an extended host-device lambda
    // Error if *this capture not enabled by language dialect
    auto lambda2 = [=, *this] __host__ __device__ { return var; };

    // Use in an unannotated lambda in a host-device function
    // Error if *this capture not enabled by language dialect
    auto lambda3 = [=, *this] { return var; };
}
};

```

5.3.8.7 Argument Dependent Lookup (ADL)

As previously mentioned, the CUDA compiler replaces an extended lambda expression with a placeholder type before invoking the host compiler. One template argument of the placeholder type uses the address of the function that encloses the original lambda expression. This may cause additional namespaces to participate in [Argument-Dependent Lookup \(ADL\)](#) for any host function call whose argument types involve the closure type of the extended lambda expression. Consequently, an incorrect function may be selected by the host compiler.

Example:

```

namespace N1 {

struct MyStruct {};

template <typename T>
void my_function(T);

}; // namespace N1

namespace N2 {

template <typename T>
int my_function(T);

template <typename T>

```

(continues on next page)

(continued from previous page)

```

void run(T in) { my_function(in); }

} // namespace N2

void bar(N1::MyStruct in) {
    // For extended device-only lambda, the code sent to the host compiler is
    →replaced with
    // the placeholder type instantiation expression
    // ' __nv_dl_wrapper_t< __nv_dl_tag<void (*)(N1::MyStruct in),(&bar),1>
    →> { }'
    //
    // As a result, the namespace 'N1' participates in ADL lookup of the
    // call to "my_function()" in the body of N2::run, causing ambiguity.
    auto lambda1 = [=] __device__ { };
    N2::run(lambda1);
}

```

In the above example, the CUDA compiler replaced the extended lambda with a placeholder type involving the N1 namespace. Consequently, the N1 namespace participates in the ADL lookup for `my_function(in)` in the body of `N2::run()`, resulting in a host compilation failure due to the discovery of multiple overload candidates: `N1::my_function` and `N2::my_function`.

5.3.9. Polymorphic Function Wrappers

The `nvfunctional` header provides a polymorphic function wrapper class template, `nvstd::function`. Instances of this class template can store, copy, and invoke any callable target, such as lambda expressions. `nvstd::function` can be used in both host and device code.

Example:

```

#include <nvfunctional>

__host__          int host_function()          { return 1; }
__device__        int device_function()        { return 2; }
__host__ __device__ int host_device_function() { return 3; }

__global__ void kernel(int* result) {
    nvstd::function<int()> fn1 = device_function;
    nvstd::function<int()> fn2 = host_device_function;
    nvstd::function<int()> fn3 = [](){ return 10; };
    *result
        = fn1() + fn2() + fn3();
}

__host__ __device__ void host_device_test(int* result) {
    nvstd::function<int()> fn1 = host_device_function;
    nvstd::function<int()> fn2 = [](){ return 10; };
    *result
        = fn1() + fn2();
}

__host__ void host_test(int* result) {
    nvstd::function<int()> fn1 = host_function;
    nvstd::function<int()> fn2 = host_device_function;

```

(continues on next page)

(continued from previous page)

```

nvstd::function<int()> fn3 = [](){ return 10; };
*result                    = fn1() + fn2() + fn3();
}

```

Invalid cases:

- ▶ Instances of `nvstd::function` in host code cannot be initialized with the address of a `__device__` function or with a functor whose `operator()` is a `__device__` function.
- ▶ Similarly, instances of `nvstd::function` in device code cannot be initialized with the address of a `__host__` function or with a functor whose `operator()` is a `__host__` function.
- ▶ `nvstd::function` instances cannot be passed from host code to device code (or vice versa) at runtime.
- ▶ `nvstd::function` cannot be used in the parameter type of a `__global__` function if the `__global__` function is launched from host code.

Examples of invalid cases:

```

#include <nvfunctional>

__device__ int device_function() { return 1; }
__host__   int host_function() { return 3; }
auto       lambda_host = [] { return 0; };

__global__ void k() {
    nvstd::function<int()> fn1 = host_function; // ERROR, initialized with
    ↪address of __host__ function
    nvstd::function<int()> fn2 = lambda_host;   // ERROR, initialized with
    ↪address of functor with
                                                //      __host__ operator()
    ↪function
}

__global__ void kernel(nvstd::function<int()> f1) {}

void foo(void) {
    auto lambda_device = [=] __device__ { return 1; };

    nvstd::function<int()> fn1 = device_function; // ERROR, initialized with
    ↪address of __device__ function
    nvstd::function<int()> fn2 = lambda_device;   // ERROR, initialized with
    ↪address of functor with
                                                //      __device__
    ↪operator() function
    kernel<<<1, 1>>>(fn2);                       // ERROR, passing
    ↪nvstd::function from host to device
}

```

`nvstd::function` is defined in the `nvfunctional` header as follows:

```

namespace nvstd {

template <typename RetType, typename ...ArgTypes>
class function<RetType(ArgTypes...)> {
public:
    // constructors
    __device__ __host__ function() noexcept;
    __device__ __host__ function(nullptr_t) noexcept;
    __device__ __host__ function(const function&);
    __device__ __host__ function(function&&);

    template<typename F>
    __device__ __host__ function(F);

    // destructor
    __device__ __host__ ~function();

    // assignment operators
    __device__ __host__ function& operator=(const function&);
    __device__ __host__ function& operator=(function&&);
    __device__ __host__ function& operator=(nullptr_t);
    template<typename F>
    __device__ __host__ function& operator=(F&&);

    // swap
    __device__ __host__ void swap(function&) noexcept;

    // function capacity
    __device__ __host__ explicit operator bool() const noexcept;

    // function invocation
    __device__ RetType operator()(ArgTypes...) const;
};

// null pointer comparisons
template <typename R, typename... ArgTypes>
__device__ __host__
bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;

template <typename R, typename... ArgTypes>
__device__ __host__
bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

template <typename R, typename... ArgTypes>
__device__ __host__
bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;

template <typename R, typename... ArgTypes>
__device__ __host__
bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

// specialized algorithms
template <typename R, typename... ArgTypes>

```

(continues on next page)

(continued from previous page)

```
__device__ __host__
void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&);

} // namespace nvstd
```

5.3.10. C/C++ Language Restrictions

5.3.10.1 Unsupported Features

- ▶ Run-Time Type Information (RTTI) and exceptions are not supported in device code:
 - ▶ typeid keyword
 - ▶ dynamic_cast keyword
 - ▶ try/catch/throw keywords
- ▶ long double is not supported in device code.
- ▶ Trigraphs are not supported on any platform. Digraphs are not supported on Windows.
- ▶ User-defined operator new, operator new[], operator delete, or operator delete[] cannot be used to replace the corresponding built-ins provided by the compiler, and it is considered undefined behavior on both host and device.

5.3.10.2 Namespace Reservations

Unless otherwise noted, adding definitions to top-level namespaces `cuda::`, `nv::`, or `cooperative_groups::`, or to any nested namespace within them, is undefined behavior. We allow `cuda::` as a subnamespace as depicted below:

Examples:

```
namespace cuda { // same for "nv" and "cooperative_groups" namespaces

struct foo;      // ERROR, class declaration in the "cuda" namespace

void bar();      // ERROR, function declaration in the "cuda" namespace

namespace utils {} // ERROR, namespace declaration in the "cuda" namespace

} // namespace cuda
```

```
namespace utils {
namespace cuda {

// CORRECT, namespace "cuda" may be used nested within a non-reserved namespace
void bar();

} // namespace cuda
} // namespace utils

// ERROR, Equivalent to adding symbols to namespace "cuda" at global scope
using namespace utils;
```

5.3.10.3 Pointers and Memory Addresses

Pointer dereferencing (`*pointer`, `pointer->member`, `pointer[0]`) is allowed only in the same execution space where the associated memory resides. The following cases result in undefined behavior, most often a segmentation fault and application termination.

- Dereferencing a pointer either to *global memory*, *shared memory*, or *constant memory* on the host.
- Dereferencing a pointer to host memory in device code.

The following restrictions apply to functions:

- It is not allowed to take the address of a `__device__` function in host code.
- The address of a `__global__` function taken in host code cannot be used in device code. Similarly, the address of a `__global__` function taken in device code cannot be used in host code.

The address of a `__device__` or `__constant__` variable obtained through `cudaGetSymbolAddress()` as described in the *Memory Space Specifiers* section can only be used in host code.

5.3.10.4 Variables

5.3.10.4.1 Local Variables

The `__device__`, `__tile__`, `__shared__`, `__managed__`, and `__constant__` memory space specifiers are not allowed on non-extern variable declarations within a function that executes on the host.

Examples:

```
__host__ void host_function() {
    int x;                // CORRECT, __host__ variable
    __device__ int y;     // ERROR, __device__ variable declaration
    ↪within a host function
    __tile__ int z;       // ERROR, __tile__ variable declaration within
    ↪a host function
    __shared__ int w;     // ERROR, __shared__ variable declaration
    ↪within a host function
    __managed__ int h;    // ERROR, __managed__ variable declaration
    ↪within a host function
    __constant__ int i;   // ERROR, __constant__ variable declaration
    ↪within a host function
    extern __device__ int j; // CORRECT, extern __device__ variable
}
```

The `__device__`, `__tile__`, `__constant__`, and `__managed__` memory space specifiers are not allowed on variable declarations that are neither extern nor static within a function that executes on the device.

```
__device__ void device_function() {
    int x;                // CORRECT, __device__ variable
    __constant__ int y;   // ERROR, __constant__ variable declaration
    ↪within a device function
    __managed__ int z;    // ERROR, __managed__ variable declaration
    ↪within a device function
    extern __device__ int k; // CORRECT, extern __device__ variable
}
```

see also the *static variables* section.

5.3.10.4.2 const-qualified Variables

A const-qualified variable without memory space annotations (`__device__`, `__tile__`, or `__constant__`) declared at global, namespace, or class scope is considered to be a host variable. Device code cannot contain a reference or take the address of the variable.

The variable may be directly used in device code, if

- ▶ it has been initialized with a constant expression before the point of use,
- ▶ the type is not `volatile`-qualified, and
- ▶ it has one of the following types:
 - ▶ built-in integral type, or
 - ▶ built-in floating point type, except when the host compiler is Microsoft Visual Studio.

Starting with C++14, it is recommended to use `constexpr` or `inline constexpr` (C++17) variables instead of const-qualified ones. `constexpr` variables are not subject to the same type restrictions and can be utilized directly in device code.

`__managed__` variables don't support const-qualified types.

Examples:

```
const          int    ConstVar          = 10;
const          float  ConstFloatVar     = 5.0f;
inline constexpr float  ConstexprFloatVar = 5.0f; // C++17

struct MyStruct {
    static const          int    ConstVar          = 20;
    // static const          float ConstFloatVar     = 5.0f; // ERROR, static
    // → const variables cannot be float
    static inline constexpr float ConstexprFloatVar = 5.0f; // CORRECT
};

extern const int ExternVar;

__device__ void foo() {
    int array1[ConstVar];           // CORRECT
    int array2[MyStruct::ConstVar]; // CORRECT

    const    float var1 = ConstFloatVar; // CORRECT, except when the
    // → host compiler is Microsoft Visual Studio.
    constexpr float var2 = ConstexprFloatVar; // CORRECT
    // int          var3 = ExternVar;           // ERROR, "ExternVar" is not
    // → initialized with a constant expression
    // int&          var4 = ConstVar;           // ERROR, reference to host
    // → variable
    // int*          var5 = &ConstVar;         // ERROR, address of host
    // → variable
}
```

See the example on [Compiler Explorer](#).

5.3.10.4.3 volatile-qualified Variables

Note

The `volatile` keyword is supported to maintain compatibility with ISO C++. However, few, if any, of its [remaining non-deprecated uses](#) apply to GPUs.

Reading and writing to volatile-qualified objects are not atomic and are compiled into one or more [volatile instructions](#) that do not guarantee:

- ▶ ordering of memory operations, or
- ▶ that the number of memory operations performed by the hardware matches the number of PTX instructions.

In tile code, the `volatile` keyword has no effect on the behavior of memory accesses.

CUDA C++ `volatile` is NOT suitable for:

- ▶ **Inter-Thread Synchronization:** Use atomic operations via `cuda::atomic_ref`, `cuda::atomic`, or [Atomic Functions](#) instead.

Atomic memory operations provide inter-thread synchronization guarantees and deliver better performance than volatile operations. However, CUDA C++ `volatile` operations do not provide any inter-thread synchronization guarantees and are therefore not suitable for this purpose. The following example shows how to pass a message between two threads using atomic operations.

`cuda::atomic_ref`

```
#include <cuda/atomic>

__global__ void kernel(int* flag, int* data) {
    cuda::atomic_ref<int, cuda::thread_scope_device> atomic_ref{*flag};
    if (threadIdx.x == 0) {
        // Consumer: blocks until flag is set by producer, then reads
        ↪data
        while(atomic_ref.load(cuda::memory_order_acquire) == 0)
            ;
        if (*data != 42)
            __trap(); // Errors if wrong data read
    }
    else if (threadIdx.x == 1) {
        // Producer: writes data then sets flag
        *data = 42;
        atomic_ref.store(1, cuda::memory_order_release);
    }
}
```

cuda::atomic

```
#include <cuda/atomic>

__global__ void kernel(cuda::atomic<int, cuda::thread_scope_device>*
→flag, int* data) {
    if (threadIdx.x == 0) {
        // Consumer: blocks until flag is set by producer, then reads
→data
        while(flag->load(cuda::memory_order_acquire) == 0)
            ;
        if (*data != 42)
            __trap(); // Errors if wrong data read
    }
    else if (threadIdx.x == 1) {
        // Producer: writes data then sets flag
        *data = 42;
        flag->store(1, cuda::memory_order_release);
    }
}
```

Atomic Functions (atomicAdd and atomicExch)

```
__global__ void kernel(int* flag, int* data) {
    if (threadIdx.x == 0) {
        // Consumer: blocks until flag is set by producer, then reads
→data
        while(atomicAdd(flag, 0) == 0)
            ; // Load with Relaxed Read-Modify-Write
        __threadfence(); // SequentiallyConsistent fence
        if (*data != 42)
            __trap(); // Errors if wrong data read
    } else if (threadIdx.x == 1) {
        // Producer: writes data then sets flag
        *data = 42;
        __threadfence(); // SequentiallyConsistent fence
        atomicExch(flag, 1); // Store with Relaxed Read-Modify-Write
    }
}
```

- **Memory Mapped IO (MMIO):** Use **PTX MMIO operations** via inline PTX instead.

PTX MMIO operations strictly preserve the number of memory accesses performed. However, CUDA C++ volatile operations do not preserve the number of memory accesses performed and may perform more or fewer accesses than requested in an undetermined way. This makes them unsuitable for MMIO. The following example shows how to read from and write to a register using PTX MMIO operations.

```

__global__ void kernel(int* mmio_reg0, int* mmio_reg1) {
    // Write to MMIO register:
    int value = 13;
    asm volatile("st.relaxed.mmio.sys.u32 [%0], %1;"
        :
        : "l"(mmio_reg0), "r"(value) : "memory");

    // Read MMIO register:
    asm volatile("ld.relaxed.mmio.sys.u32 %0, [%1];"
        : "=r"(value)
        : "l"(mmio_reg1) : "memory");

    if (value != 42)
        __trap(); // Errors if wrong data read
}

```

5.3.10.4.4 static Variables

static local variables are allowed in device functions.

A different static variable is used for each execution space of the enclosing function. For example,

- A `__host__ __device__` function has one copy of the static variable for host execution and one copy for device execution.
- A `__tile__ __device__` function has one copy for tile execution and one copy for SIMT execution.

If the function has a `__host__` execution space specifier, static variables with an explicit memory space, such as `static __device__`/`__tile__`/`__constant__`/`__shared__`/`__managed__`, are allowed only when `__CUDA_ARCH__` is defined.

Examples of legal and illegal uses of function-scope static variables are shown below.

```

struct TrivialStruct {
    int x;
};

struct NonTrivialStruct {
    __device__ NonTrivialStruct(int x) {}
};

__device__ void device_function(int x) {
    static int v1;           // CORRECT, implicit __device__ memory space
    ↪specifier
    static int v2 = 11;      // CORRECT, implicit __device__ memory space
    ↪specifier
    // static int v3 = x;    // ERROR, dynamic initialization is not allowed

    static __managed__ int v4; // CORRECT, explicit
    static __device__ int v5;  // CORRECT, explicit
    static __constant__ int v6; // CORRECT, explicit
    static __shared__ int v7;  // CORRECT, explicit

    static TrivialStruct s1;    // CORRECT, implicit __device__ memory

```

(continues on next page)

(continued from previous page)

```

→space specifier
    static TrivialStruct    s2{22}; // CORRECT, implicit __device__ memory
→space specifier
// static TrivialStruct    s3{x};    // ERROR, dynamic initialization is not
→allowed
// static NonTrivialStruct s4{3};    // ERROR, dynamic initialization is not
→allowed
}

```

See the example on [Compiler Explorer](#).

```

__host__ __device__ void host_device_function() {
    static int v1; // CORRECT, implicit __device__ memory space
→specifier
// static __device__ int v2; // ERROR, __device__-only variable inside a host-
→device function
#ifdef __CUDA_ARCH__
    static __device__ int v3; // CORRECT, declaration is only visible during
→device compilation
#else
    static int v4;             // CORRECT, declaration is only visible during
→host compilation
#endif
}

```

See the example on [Compiler Explorer](#).

```

#include <cassert>

__host__ __device__ int host_device_function() {
    static int v = 0;
    v++;
    return v;
}

__global__ void kernel() {
    int ret = host_device_function(); // v = 1
    assert(ret == 4);                 // FAIL
}

int main() {
    host_device_function();           // v = 1
    host_device_function();           // v = 2
    int ret = host_device_function(); // v = 3
    assert(ret == 3);                 // OK
    kernel<<<1, 1>>>();
    cudaDeviceSynchronize();
}

```

See the example on [Compiler Explorer](#).

5.3.10.4.5 extern Variables

When compiling in the *whole program compilation mode*, `__device__`, `__tile__`, `__shared__`, `__managed__`, and `__constant__` variables cannot be defined with external linkage using the `extern` keyword. This restriction also applies in *separate compilation mode* for `__tile__` variables.

The only exception is for dynamically allocated `__shared__` variables as described in the *Dynamic Allocation of Shared Memory* section.

```
__device__      int x; // OK
extern __device__ int y; // ERROR in whole program compilation mode
extern __shared__ int z; // OK
```

5.3.10.5 Functions

5.3.10.5.1 Recursion

`__global__`, `__tile_global__` and `__tile__` functions do not support recursion, while `__device__` and `__host__` `__device__` functions do not have such restriction.

5.3.10.5.2 External Linkage

Device variables or functions with external linkage require *separate compilation mode* across multiple translation units.

In separate compilation mode, if a `__device__` or `__global__` function definition is required to exist in a particular translation unit, then the parameters and return types of the function must be complete in that translation unit. The concept is also known as One Definition Rule-use, or ODR-use.

Example:

```
//first.cu:
struct S;           // forward declaration
__device__ void foo(S); // ERROR, type 'S' is an incomplete type
__device__ auto* ptr = foo; // ODR-use, address taken

int main() {}
```

```
//second.cu:
struct S {};        // struct definition
__device__ void foo(S) {} //function definition
```

```
# compiler invocation
$ nvcc -std=c++14 -rdc=true first.cu second.cu -o prog
nvlink error   : Prototype doesn't match for '_Z3foo1S' in '/tmp/tmpxft_
→00005c8c_00000000-18_second.o',
                first defined in '/tmp/tmpxft_00005c8c_00000000-18_second.o'
nvlink fatal   : merge_elf failed
```

5.3.10.5.3 Formal Parameters

The `__device__`, `__tile__`, `__shared__`, `__managed__` and `__constant__` memory space specifiers are not allowed on formal parameters.

```
void device_function1(__device__ int x) { } // ERROR, __device__ parameter
void device_function2(__shared__ int x) { } // ERROR, __shared__ parameter
```

5.3.10.5.4 __global__ Function Parameters

A `__global__` or `__tile_global__` function has the following restrictions:

- ▶ It cannot have a variable number of arguments, namely the C ellipsis syntax `...` and the `va_list` type. C++11 variadic template is allowed, subject to the restrictions described in the [__global__ Variadic Template](#) section.
- ▶ Function parameters are passed to the device via *constant memory* and their total size is limited to 32,764 bytes.
- ▶ Function parameters cannot be of type `std::initializer_list`.
- ▶ Polymorphic class parameters (virtual) are considered undefined behavior.
- ▶ Lambda expressions and closure types are allowed, subject to the restrictions described in the [Lambda Expressions and __global__ Function Parameters](#) section.
- ▶ For `__tile_global__` functions, the function parameters may not be pass-by-value classes, structs, or unions.

5.3.10.5.5 __global__ Function Arguments Passing

When launching a `__global__` function *from device code*, each argument must be trivially copyable and trivially destructible.

When a `__global__` function is launched from host code, each argument type may be non-trivially copyable or non-trivially destructible. However, the processing of these types does not follow the standard C++ model, as described below. The user code must ensure that this workflow does not affect program correctness. The workflow diverges from standard C++ in two areas:

1. Raw memory copy instead of copy constructor invocation

The CUDA Runtime passes the kernel arguments to the `__global__` function by copying the raw memory content, eventually using `memcpy`. If an argument is non-trivially copyable and provides a user-defined copy constructor, the operations and side effects of the invocation are skipped in the host-to-device copy.

Example:

```
#include <cassert>

struct MyStruct {
    int value = 1;
    int* ptr;

    MyStruct() = default;

    __host__ __device__ MyStruct(const MyStruct&) { ptr = &value; }
};

__global__ void device_function(MyStruct my_struct) {
    // this assert fails because "my_struct" is obtained by copying
    // the raw memory content and the copy constructor is skipped.
    assert(&my_struct.value == &my_struct.ptr);
}
```

(continues on next page)

(continued from previous page)

```

    assert(my_struct.ptr == &my_struct.value); // FAIL
}

void host_function(MyStruct my_struct) {
    assert(my_struct.ptr == &my_struct.value); // CORRECT
}

int main() {
    MyStruct my_struct;
    host_function(my_struct);
    device_function<<<1, 1>>>(my_struct); // copy constructor invoked in
    →the host-side only
    cudaDeviceSynchronize();
}

```

See the example on [Compiler Explorer](#).

2. Destructor may be invoked before the `__global__` function has finished

Kernel launches are asynchronous with host execution. As a result, if a `__global__` function argument has a non-trivial destructor, the destructor may execute in host code even before the `__global__` function has finished execution. This may break programs where the destructor has side effects.

Example:

```

#include <cassert>

__managed__ int var = 0;

struct MyStruct {
    __host__ __device__ ~MyStruct() { var = 3; }
};

__global__ void device_function(MyStruct my_struct) {
    assert(var == 0); // FAIL, MyStruct::~~MyStruct() sets the value to 3
}

int main() {
    MyStruct my_struct;
    // GPU kernel execution is asynchronous with host execution.
    // As a result, MyStruct::~~MyStruct() could be executed before
    // the kernel finishes executing.
    device_function<<<1, 1>>>(my_struct);
    cudaDeviceSynchronize();
}

```

See the example on [Compiler Explorer](#).

5.3.10.6 Classes

5.3.10.6.1 Class-type Variables

A variable definition with `__device__`, `__tile__`, `__constant__`, `__managed__` or `__shared__` memory space cannot have a class type with a non-empty constructor or a non-empty destructor.

A constructor for a class type is considered empty if it is either trivial or satisfies all of the following conditions at a point in the translation unit:

- ▶ The constructor function has been defined.
- ▶ The constructor function has no parameters, an empty initializer list, and an empty compound statement function body.
- ▶ Its class has no `virtual` functions, `virtual` base classes, or non-static data member initializers.
- ▶ The default constructors of all of its base classes can be considered empty.
- ▶ For all non-static data members of the class that are of a class type (or an array thereof), the default constructors can be considered empty.

A class's destructor is considered empty if it is either trivial or satisfies all of the following conditions at a point in the translation unit:

- ▶ The destructor function has been defined.
- ▶ The destructor function body is an empty compound statement.
- ▶ Its class has no `virtual` functions or `virtual` base classes.
- ▶ The destructors of all of its base classes can be considered empty.
- ▶ For all non-static data members of the class that are of a class type (or an array thereof), the destructor can be considered empty.

5.3.10.6.2 Data Members

The `__device__`, `__tile__`, `__shared__`, `__managed__` and `__constant__` memory space specifiers are not allowed on `class`, `struct`, and `union` data members.

Only static data members evaluated at compile time are supported, such as *const-qualified* and `constexpr` variables.

```
struct MyStruct {
    static inline constexpr int value1 = 10; // C++17
    static constexpr      int value2 = 10; // C++11
    static const          int value3 = 10;
    // static              int value4; // ERROR
};
```

5.3.10.6.3 Function Members

`__global__` and `__tile_global__` functions cannot be members of a `struct`, `class`, or `union`.

A `__global__` or `__tile_global__` function is allowed in a friend declaration, but cannot be defined.

Example:

```
struct MyStruct {
    friend __global__ void f(); // CORRECT, friend declaration only

    // friend __global__ void g() {} // ERROR, friend definition
};
```

See the example on [Compiler Explorer](#).

5.3.10.6.4 Implicitly-Declared and Non-Virtual Explicitly-Defaulted functions

Implicitly-declared special member functions are those the compiler declares for a class when the user does not declare them; Explicitly-defaulted functions are ones the user declares but marks with `= default`. The special member functions that are implicitly-declared or explicitly-defaulted are default constructor, copy constructor, move constructor, copy assignment operator, move assignment operator, and destructor.

Let F denote a non-virtual function that is either implicitly declared or explicitly defaulted on its first declaration. The execution space specifiers for F are the union of the execution space specifiers of all functions that invoke it. Note that for this analysis, a `__global__` caller will be treated as a `__device__` caller. For example:

```
class Base {
    int x;
public:
    __host__ __device__ Base() : x(10) {}
};

class Derived : public Base {
    int y;
};

class Other: public Base {
    int z;
};

__device__ void foo() {
    Derived D1;
    Other D2;
}

__host__ void bar() {
    Other D3;
}
```

In this case, the implicitly declared constructor function `Derived::Derived()` will be treated as a `__device__` function because it is only invoked from the `__device__` function `foo()`. The implicitly declared constructor function `Other::Other()` will be treated as a `__host__ __device__` function since it is invoked both from both a `__device__` function `foo()` and a `__host__` function `bar()`.

Additionally, if F is an implicitly-declared virtual function (for example, a virtual destructor), the execution spaces of each virtual function D that is overridden by F are added to the set of execution spaces for F if D not implicitly-declared.

For example:

```
struct Base1 {
    virtual __host__ __device__ ~Base1() {}
};

struct Derived1 : Base1 {}; // implicitly-declared virtual destructor
                           // ~Derived1() has __host__ __device__ execution
                           ↪space specifiers
```

(continues on next page)

(continued from previous page)

```

struct Base2 {
    virtual __device__ ~Base2() = default;
};

struct Derived2 : Base2 {}; // implicitly-declared virtual destructor
                           // ~Derived2() has __device__ execution space
→specifiers

```

5.3.10.6.5 Polymorphic Classes

Polymorphic classes, namely those with `virtual` functions, derived from other polymorphic classes, or with polymorphic data members, are subject to the following restrictions:

- ▶ Copying polymorphic objects from device to host or from host to device, including `__global__` function arguments is undefined behavior.
- ▶ The execution space of an overridden `virtual` function must match the execution space of the function in the base class.

Example:

```

struct MyClass {
    virtual __host__ __device__ void f() {}
};

__global__ void kernel(MyClass my_class) {
    my_class.f(); // undefined behavior
}

int main() {
    MyClass my_class;
    kernel<<<1, 1>>>(my_class);
    cudaDeviceSynchronize();
}

```

See the example on [Compiler Explorer](#).

```

struct BaseClass {
    virtual __host__ __device__ void f() {}
};

struct DerivedClass : BaseClass {
    __device__ void f() override {} // ERROR
};

```

See the example on [Compiler Explorer](#).

5.3.10.6.6 Windows-Specific Class Layout

The CUDA compiler follows the IA64 ABI for class layout, while Microsoft Visual Studio does not. This prevents bitwise copy of special objects between host and device code as described below.

Let T denote a pointer to member type, or a class type that satisfies any of the following conditions:

- ▶ T is a *polymorphic class*
- ▶ T has multiple inheritance with more than one direct or indirect *empty base class*.
- ▶ All direct and indirect base classes B are *empty* and the type of the first field F of T uses B in its definition, such that B is laid out at offset 0 in the definition of F.

Classes of type T, with a base class of type T, or with data members of type T, may have a different class layout and size between host and device when compiled with Microsoft Visual Studio.

Copying such objects from device to host or from host to device, including `__global__` function arguments is undefined behavior.

5.3.10.7 Templates

A type cannot be used as template argument of a `__global__` function or a `__device__`/`__constant__` variable (C++14) if either:

- ▶ The type is defined within a `__host__` or `__host__ __device__` function scope.
- ▶ The type is unnamed, such as an anonymous struct or a lambda expression, unless the type is local to a `__device__` or `__global__` function.
- ▶ The type is a class member with `private` or `protected`, unless the class is local to a `__device__` or `__global__` function.
- ▶ The type is compounded from any of the types above.

Example:

```
template <typename T>
__global__ void kernel() {}

template <typename T>
__device__ int device_var; // C++14

struct {
    int v;
} unnamed_struct;

void host_function() {
    struct LocalStruct {};
    // kernel<LocalStruct><<<1, 1>>>(); // ERROR, LocalStruct is defined within a
    ↪ host function
    int data = 4;
    // cudaMemcpyToSymbol(device_var<LocalStruct>, &data, sizeof(data)); // ERROR,
    ↪ same as above

    auto lambda = [](){};
    // kernel<decltype(lambda)><<<1, 1>>>(); // ERROR, unnamed type
    // kernel<decltype(unnamed_struct)><<<1, 1>>>(); // ERROR, unnamed type
}

class MyClass {
private:
    struct PrivateStruct {};
public:
    static void launch() {
```

(continues on next page)

(continued from previous page)

```
//      kernel<PrivateStruct><<<1, 1>>>(); // ERROR, private type
//    }
};
```

See the example on [Compiler Explorer](#).

5.3.10.8 Restrictions in Tile Code

Functions annotated with `__tile__` or `__tile_global__` have the following additional restrictions:

- ▶ The language constructs below are unsupported in tile code:
 - ▶ Return statements inside `do`, `while` or `for` loops.
 - ▶ Virtual function invocations.
 - ▶ `Goto` statements.
 - ▶ `Switch` statements.
 - ▶ Expressions yielding a function pointer, function reference, pointer to member variable, or pointer to member function.
 - ▶ Function pointer and pointer to member function invocations.
 - ▶ Pointer to member variable accesses.
 - ▶ 128-bit integer or floating point types.
 - ▶ Types containing bitfields.
 - ▶ Types exceeding 16 MB in size.
 - ▶ Types with virtual base classes or virtual functions.
 - ▶ Dynamic memory allocation or deallocation with non-placement operator `new` or `delete`.
- ▶ A `__tile__` or `__tile_global__` function must have a function body in the same translation unit in which it is declared.
- ▶ Annotating a virtual function with `__tile__` is unsupported.
- ▶ A `__tile_global__` or `__tile__` function may not have a variable number of arguments using the C ellipsis syntax `...`.
- ▶ A `__tile_global__` or `__tile__` function may not be directly or indirectly recursive.
- ▶ For `__tile_global__` functions, the function parameters may not be pass-by-value classes, structs, or unions.
- ▶ Tile code may not perform device-side kernel launches and tile kernels may not be launched from device side kernel invocations.
- ▶ Direct access to the `__x` member variable of `__half`, `__nv_bfloat16` and related extended floating point types is unsupported in tile code.

5.3.11. C++11 Restrictions

5.3.11.1 inline Namespaces

It is not allowed to define one of the following entities within an `inline` namespace when another entity of the same name and type signature is defined in an enclosing namespace:

- ▶ `__global__` or `__tile_global__` functions.
- ▶ `__device__`, `__tile__`, `__constant__`, `__managed__`, `__shared__` variables.
- ▶ Variables with surface or texture type, such as `cudaSurfaceObject_t` or `cudaTextureObject_t`.

Example:

```
__device__ int my_var; // global scope

inline namespace NS {

__device__ int my_var; // namespace scope

} // namespace NS
```

5.3.11.2 inline Unnamed Namespaces

The following entities cannot be declared in namespace scope within an inline unnamed namespace:

- ▶ `__global__` or `__tile_global__` function.
- ▶ `__device__`, `__tile__`, `__constant__`, `__managed__`, `__shared__` variables.
- ▶ Variables with surface or texture type, such as `cudaSurfaceObject_t` or `cudaTextureObject_t`.

5.3.11.3 constexpr Functions

A `__global__` function cannot be declared as `constexpr`. By default, a `constexpr` function cannot be called from a function with incompatible execution space, in the same way as standard functions. In the example code listed in this section UB stands for ‘undefined behavior’.

- ▶ Calling a `constexpr` function that does not have an explicit or implicit `__host__` annotation from a host function during host compilation phase (when `__CUDA_ARCH__` macro is undefined) has undefined behavior. Example:

```
constexpr __device__ int device_func() { return 0; }
constexpr __tile__ int tile_func() { return 0; }

constexpr __device__ __host__ int host_device_func() { return
→0; }

int main() {
    constexpr int x1 = device_func(); // UB: calling a __device__-
→only constexpr function from host code
    constexpr int x2 = tile_func(); // UB: calling a __tile__-
→only constexpr function from host code
    constexpr int x3 = host_device_func(); // OK
}
```

- ▶ Calling a `constexpr` function that does not have an explicit or implicit `__device__` annotation from a `__device__` or `__global__` function during device compilation phase (when `__CUDA_ARCH__` macro is defined) has undefined behavior. Example:

```
constexpr int host_func() { return 0; }

__device__ void dmain()
{
    int x = host_func(); // UB: calling a host-only constexpr
    ↪function from device code
}
```

- Calling a constexpr function that does not have an explicit or implicit `__tile__` annotation from a `__tile__` or `__tile_global__` function during device compilation phase (when `__CUDA_ARCH__` macro is defined) has undefined behavior. Example:

```
constexpr int host_func() { return 0; }

__tile__ void dmain()
{
    int x = host_func(); // UB: calling a host-only constexpr
    ↪function from tile code
}
```

Note that a function template specialization may not be a constexpr function even if the corresponding template function is marked with the keyword `constexpr`.

Relaxed constexpr-Function Support

The experimental nvcc flag `--expt-relaxed-constexpr` can be used to relax this constraint as described below. nvcc will also define the macro `__CUDACC_RELAXED_CONSTEXPR__`. In the example code listed in this section UB stands for ‘undefined behavior’.

When the `--expt-relaxed-constexpr` flag is specified, the compiler will support cross execution space calls, as follows:

1. A cross-execution space call to a constexpr function is supported if it occurs in a context that requires constant evaluation, such as in the initializer of a constexpr variable. Example:

```
constexpr __host__ int host_func(int x) { return x + 1; };

__global__ void doit() {
    constexpr int val = host_func(1); // OK: call is in a context
    ↪that                                     // requires constant
    ↪evaluation.
}

__tile_global__ void tile_doit() {
    constexpr int val = host_func(1); // OK: call is in a context
    ↪that                                     // requires constant
    ↪evaluation.
}

constexpr __device__ int device_func(int x) { return x + 1; }

constexpr __tile__ int tile_func(int x) { return x + 2; }
```

(continues on next page)

(continued from previous page)

```

int main() {
constexpr int val = device_func(1) + tile_func(1); // OK: call is
↳in a context that
                                                    // requires
↳constant evaluation.
}

```

2. Otherwise:

- a. A cross-execution space call from Tile code to a constexpr function that does not have an explicit or implicit `__tile__` annotation is not supported outside of a context where constant folding is required by the language rules. Example:

```

constexpr __host__ int host_func(int x) { return x + 1; }

__tile__ int doit(int in) {
    in = host_func(in); // UB: call occurs outside of a
↳context that requires
                        // constant evaluation.
    constexpr int other = host_func(10); // OK with -expt-
↳relaxed-constexpr:
                                                    // call is
↳required to be evaluated at compile time
}

```

- b. During SIMT device code generation, device code is generated for the body of a host-only constexpr function `host_func`, unless `host_func` is not used or is only called in a constant evaluation context. Example:

```

// NOTE: "host_func" is emitted in generated device code
↳because it is
// called from device code in a non-constexpr context
constexpr __host__ int host_func(int x) { return x + 1; }

__device__ int doit(int in) {
    in = host_func(in); // OK, even though argument is
↳not a constant expression
    return in;
}

```

- c. All code restrictions applicable to a `__device__` function are also applicable to the constexpr host-only function `H` that is called from SIMT device code. However, compiler may not emit any build time diagnostics for `H` for these restrictions. The reason is that diagnostics are usually generated during parsing, but `H` may already have been parsed before the call to `H` from device code is encountered later in the translation unit.

For example, the following code patterns are unsupported in the body of `H` (as with any `__device__` function), but no compiler diagnostic may be generated:

- ODR-use of a host variable or host-only non-constexpr function. Example:

```

int host_var1, host_var2;

constexpr __host__ int* host_func(bool b) { return b ? &
↳host_var1 : &host_var2; };

__device__ int doit(bool flag) {
    int *ptr;
    ptr = host_func(flag); // UB: host_func() attempts to
↳refer to host variables 'host_var1' and 'host_var2'.
                           // code will compile, but will NOT
↳execute correctly.
    return *ptr;
}

```

- Use of exceptions (throw/catch) and RTTI (typeid, dynamic_cast). Example:

```

struct Base { };
struct Derived : public Base { };

// NOTE: "host_func" is emitted in generated device code
constexpr int host_func(bool b, Base *ptr) {
    if (b) {
        return 1;
    } else if (typeid(ptr) == typeid(Derived)) { // UB:
↳use of typeid in code executing on the GPU
        return 2;
    } else {
        throw int{4}; // UB: use of throw in code executing
↳on the GPU
    }
}

__device__ void doit(bool flag) {
    int val;
    Derived d;
    val = host_func(flag, &d); //UB: host_func() attempts
↳use typeid and throw(), which are not allowed in code
↳that executes on the GPU
}

```

- d. During host code generation, the body of a constexpr non-host function F is preserved in the code sent to the host compiler. If the body of F attempts to ODR-use a namespace scope device or `__tile__` variable or a non-host non-constexpr function, then the call to F from host code is not supported (code may build without compiler diagnostics, but may behave incorrectly at run time). Example:

```

__device__ int device_var1, device_var2;

constexpr __device__ int* device_func(bool b) { return b ? &
↳device_var1 : &device_var2; };

__tile__ int tile_var1, tile_var2;

```

(continues on next page)

(continued from previous page)

```
constexpr __tile__ int* tile_func(bool b) { return b ? &tile_
    ↪var1 : &tile_var2; };

int doit1(bool flag) {
    int *ptr;
    ptr = device_func(flag); // UB: device_func() attempts to
    ↪refer to device variables 'device_var1' and 'device_var2'
                                // code will compile, but will NOT
    ↪execute correctly.
    return *ptr;
}

int doit2(bool flag) {
    int *ptr;
    ptr = tile_func(flag); // UB: tile_func() attempts to refer
    ↪to __tile__ variables 'tile_var1' and 'tile_var2'
                                // code will compile, but will NOT
    ↪execute correctly.
    return *ptr;
}
```

Warning

Due to the above restrictions and the lack of compiler diagnostics for incorrect usage, it is recommended to avoid calling a function in the Standard C++ headers `std::` from device code. The implementation of such functions varies depending on the host platform. Instead, it is strongly suggested to call the equivalent functionality in the CUDA C++ Standard Library *libcu++*, in the `cuda::std::` namespace.

5.3.11.4 constexpr Variables

By default, a `constexpr` variable cannot be used in a function with incompatible execution space, in the same way of standard variables.

A `constexpr` variable can be directly used in device code in the following cases:

- ▶ C++ scalar types, excluding pointer and pointer-to-member types:
 - ▶ `nullptr_t`.
 - ▶ `bool`.
 - ▶ Integral types: `char`, `signed char`, `unsigned char`, `long`, `long long`, etc.
 - ▶ Floating point types: `float`, `double`.
 - ▶ Enumerators: `enum` and `enum class`.
- ▶ Class types: `class`, `struct`, and `union` with a `constexpr` constructor.
- ▶ Raw array of the types above, for example `int[]`, only when they are used inside a `constexpr __device__` or `__host__ __device__` function.

`constexpr __managed__` and `constexpr __shared__` variables are not allowed.

Examples:

```
constexpr int ConstexprVar = 4; // scalar type

struct MyStruct {
    static constexpr int ConstexprVar = 100;
};

constexpr MyStruct my_struct = MyStruct{}; // class type

constexpr int array[] = {1, 2, 3};

__device__ constexpr int get_value(int idx) {
    return array[idx]; // CORRECT
}

__device__ void foo(int idx) {
    int v1 = ConstexprVar; // CORRECT
    int v2 = MyStruct::ConstexprVar; // CORRECT
    // const int &v3 = ConstexprVar1; // ERROR, reference to host
    →constexpr variable
    // const int *v4 = &ConstexprVar1; // ERROR, address of host constexpr
    →variable
    int v5 = get_value(2); // CORRECT, 'get_value(2)' is a
    →constant expression.
    // int v6 = get_value(idx); // ERROR, 'get_value(idx)' is not a
    →constant expression
    // int v7 = array[2]; // ERROR, 'array' is not scalar
    →type.
    MyStruct v8 = my_struct; // CORRECT
}
```

See the example on [Compiler Explorer](#).

5.3.11.5 __global__ Variadic Template

A variadic `__global__` or `__tile_global__` function template has the following restrictions:

- Only a single pack parameter is allowed.
- The pack parameter must be listed last in the template parameter list.

Examples:

```
template <typename... Pack>
__global__ void kernel1(); // CORRECT

// template <typename... Pack, template T>
// __global__ void kernel2(); // ERROR, parameter pack is not the last parameter

template <typename... TArgs>
struct MyStruct {};

// template <typename... Pack1, typename... Pack2>
// __global__ void kernel3(MyStruct<Pack1...>, MyStruct<Pack2...>); // ERROR,
→more than one parameter pack
```

See the example on [Compiler Explorer](#).

5.3.11.6 Defaulted Functions = default

The CUDA compiler infers the execution space of explicitly-defaulted member functions as described in *Implicitly-declared and explicitly-defaulted functions*.

Execution space specifiers on explicitly-defaulted functions are ignored by the compiler, except in the case the function is defined out-of-line or is a virtual function.

Examples:

```
struct MyStruct1 {
    MyStruct1() = default;
};

void host_function() {
    MyStruct1 my_struct; // __host__ __device__ constructor
}

__device__ void device_function() {
    MyStruct1 my_struct; // __host__ __device__ constructor
}

struct MyStruct2 {
    __device__ MyStruct2() = default; // WARNING: __device__ annotation is
    ↪ignored
};

struct MyStruct3 {
    __host__ MyStruct3();
};
MyStruct3::MyStruct3() = default; // out-of-line definition, not ignored

__device__ void device_function2() {
    // MyStruct3 my_struct; // ERROR, __host__ constructor
}

struct MyStruct4 {
    // MyStruct4::~MyStruct4 has host execution space, not ignored because
    ↪virtual
    virtual __host__ ~MyStruct4() = default;
};

__device__ void device_function3() {
    MyStruct4 my_struct4;
    // implicit destructor call for 'my_struct4':
    // ERROR: call from a __device__ function 'device_function3' to a
    // __host__ function 'MyStruct4::~MyStruct4'
}
```

See the example on [Compiler Explorer](#).

5.3.11.7 [cuda::]std::initializer_list

By default, the CUDA compiler implicitly considers the member functions of [cuda::]std::initializer_list to have __host__ __device__ __tile__ execution space specifiers, and therefore they can be invoked directly from device code.

The nvcc flag --no-host-device-initializer-list disables this behavior; member functions of [cuda::]std::initializer_list will then be considered as __host__ functions and will not be directly invocable from device code.

A __global__ or __tile_global__ function cannot have a parameter of type [cuda::]std::initializer_list.

Example:

```
#include <initializer_list>

__device__ void foo(std::initializer_list<int> in) {}

__device__ void bar() {
    foo({4,5,6}); // (a) initializer list containing only constant expressions.
    int i = 4;
    foo({i,5,6}); // (b) initializer list with at least one non-constant
    ↪ element.
                // This form may have better performance than (a).
}
```

See the example on [Compiler Explorer](#).

5.3.11.8 [cuda::]std::move, [cuda::]std::forward

By default, the CUDA compiler implicitly considers std::move and std::forward function templates to have __host__ __device__ __tile__ execution space specifiers, and therefore they can be invoked directly from device code. The nvcc flag --no-host-device-move-forward disables this behavior; std::move and std::forward will then be considered as __host__ functions and will not be directly invocable from device code.

Hint

cuda::std::move and cuda::std::forward on the contrary always have __host__ __device__ execution space.

5.3.12. C++ 14 Restrictions

5.3.12.1 Functions with Deduced Return Type

A __global__ or __tile_global__ function cannot have a deduced return type auto.

Introspection of the return type of a __device__ function with a deduced return type is not allowed in host code.

Note

The CUDA frontend compiler changes the function declaration to have a void return type, before invoking the host compiler. This may break introspection of the deduced return type of the `__device__` function in host code. Thus, the CUDA compiler will issue a compile-time error for referencing such a deduced return type outside of device function bodies.

Examples:

```
__device__ auto device_function(int x) { // deduced return type
    return x;                          // decltype(auto) has the same
    ↪behavior
}

__global__ void kernel() {
    int x = sizeof(device_function(2)); // CORRECT, device code scope
}

// const int size = sizeof(device_function(2)); // ERROR, return type
↪deduction on host

void host_function() {
    // using T = decltype(device_function(2)); // ERROR, return type
    ↪deduction on host
}

void host_fn1() {
    // ERROR, referenced outside device function bodies
    int (*p1)(int) = fn1;

    struct S_local_t {
        // ERROR, referenced outside device function bodies
        decltype(fn2(10)) m1;

        S_local_t() : m1(10) { }
    };
}

// ERROR, referenced outside device function bodies
template <typename T = decltype(fn2)>
void host_fn2() { }

template<typename T> struct MyStruct { };

// ERROR, referenced outside device function bodies
struct S1_derived_t : MyStruct<decltype(fn1)> { };
```

5.3.12.2 Variable Templates

A `__device__`, `__tile__` or `__constant__` variable template cannot be const-qualified when using the Microsoft compiler.

Examples:

```
// ERROR on Windows (non-portable), const-qualified
template <typename T>
__device__ const T var = 0;

// CORRECT, ptr1 is not const-qualified
template <typename T>
__device__ const T* ptr1 = nullptr;

// ERROR on Windows (non-portable), ptr2 is const-qualified
template <typename T>
__device__ const T* const ptr2 = nullptr;
```

See the example on [Compiler Explorer](#).

5.3.13. C++17 Restrictions

5.3.13.1 inline Variables

In a single translation unit, using an inline variable provides no additional functionality beyond a regular variable and does not provide any practical advantage.

nvcc allows inline variables with `__device__`, `__tile__`, `__constant__`, or `__managed__` memory space only in *Separate Compilation* mode or for variables with internal linkage.

Note

When using gcc/g++ host compiler, an inline variable declared with `__managed__` memory space specifier may not be visible to the debugger.

Examples:

```
inline __device__ int device_var1; // CORRECT, when compiled in
    ↪ Separate Compilation mode (-rdc=true or -dc)
    // ERROR, when compiled in Whole
    ↪ Program Compilation mode

static inline __device__ int device_var2; // CORRECT, internal linkage

namespace {

inline __device__ int device_var3; // CORRECT, internal linkage

inline __shared__ int shared_var; // CORRECT, internal linkage

static inline __device__ int device_var4; // CORRECT, internal linkage
```

(continues on next page)

(continued from previous page)

```
inline __device__ int device_var5;           // CORRECT, internal linkage
} // namespace
```

See the example on [Compiler Explorer](#).

5.3.13.2 Structured Binding

A structured binding cannot be declared with a memory space specifier, such as `__device__`, `__tile__`, `__shared__`, `__constant__`, or `__managed__`.

Example:

```
struct S {
    int x, y;
};
// __device__ auto [a, b] = S{4, 5}; // ERROR
```

5.3.14. C++20 Restrictions

5.3.14.1 Three-way Comparison Operator

The three-way comparison operator (`<=>`) is supported in `__device__` and `__global__` functions, but some uses implicitly rely on functionality from the C++ Standard Library, which is provided by the host implementation. Using those operators may require specifying the flag `--expt-relaxed-constexpr` to silence warnings, and the functionality requires the host implementation to satisfy the requirements of the device code.

Examples:

```
#include <compare> // std::strong_ordering implementation

struct S {
    int x, y;

    auto operator<=>(const S&) const = default; // (a)

    __host__ __device__ bool operator<=>(int rhs) const { return false; } //
    ↪(b)
};

__host__ __device__ bool host_device_function(S a, S b) {
    if (a <=> 1) // CORRECT, calls a user-defined host-device overload (b)
        return true;
    return a < b; // CORRECT, call to an implicitly-declared function (a)
                  // Note: it requires a device-compatible std::strong_ordering
                  //       implementation provided in the header <compare>
                  //       and the flag --expt-relaxed-constexpr
}
```

See the example on [Compiler Explorer](#).

5.3.14.2 consteval Functions

`constexpr` functions can be called from both host and device code, independently of their execution space.

Examples:

```
constexpr int host_consteval() {
    return 10;
}

__device__ constexpr int device_consteval() {
    return 10;
}

__device__ int device_function() {
    return host_consteval(); // CORRECT, even if called from device code
}

__host__ __device__ int host_device_function() {
    return device_function(); // CORRECT, even if called from host-device code
}
```

5.3.15. C++23 Restrictions

There are no known C++23-specific restrictions beyond the unsupported or not-applicable features indicated in the [C++23 Language Features table](#) above. Entries marked × or N/A in that table (including Defect Report resolutions) reflect missing or not-applicable features rather than additional behavioral restrictions.

5.3.15.1 Equality Operator (P2468R2)

Although NVCC does not fully implement **DR: The Equality Operator You Are Looking For** (P2468R2), it approximates the behavior and this approximation has not led to known failures in user code.

5.4. C/C++ Language Extensions

5.4.1. Function and Variable Annotations

5.4.1.1 Execution Space Specifiers

The execution space specifiers `__host__`, `__device__`, `__tile__`, `__global__`, and `__tile_global__` indicate whether a function executes in the host, SIMT, or tile context.

Table 39: Execution Space Specifier

Execution Space Specifier	Executed in			Callable from		
	Host	SIMT	Tile	Host	SIMT	Tile
<code>__host__</code> , no specifier	✓	×	×	✓	×	×
<code>__device__</code>	×	✓	×	×	✓	×
<code>__global__</code>	×	✓	×	✓	✓	×
<code>__tile__</code>	×	×	✓	×	×	✓
<code>__tile_global__</code>	×	×	✓	✓	×	×
<code>__host__</code> <code>__device__</code> <code>__tile__</code>	✓	✓	✓	✓	✓	✓

Constraints for `__global__` and `__tile_global__` functions:

- ▶ Must return void.
- ▶ Cannot be a member of a class, struct, or union.
- ▶ Requires an execution configuration as described in [Kernel Configuration](#).
- ▶ Does not support recursion.
- ▶ Refer to `__global__` [function parameters](#) for additional restrictions.

Calls to a `__global__` and `__tile_global__` function are asynchronous. They return to the host thread before the device completes execution.

Functions declared with multiple execution spaces (for example, `__host__ __device__`) are compiled for each context. The `__CUDA_ARCH__` [macro](#) can be used to differentiate the host and device code paths:

```
__host__ __device__ void func() {
#ifdef __CUDA_ARCH__
    // Device code path
#else
    // Host code path
#endif
}
```

5.4.1.2 Memory Space Specifiers

The memory space specifiers `__device__`, `__tile__`, `__managed__`, `__constant__`, and `__shared__` indicate the storage location of a variable on the device.

The following table summarizes the memory space properties:

Table 40: Memory Space Specifier

Memory Space Specifier	Location	Accessible by	Lifetime	Unique instance
<code>__device__</code>	Device memory	global Device Threads (grid) / CUDA Runtime API	Program/ <i>CUDA context</i>	Per device
<code>__tile__</code>	Device memory	global Tile blocks / CUDA Runtime API	Program/ <i>CUDA context</i>	Per device
<code>__constant__</code>	Device memory	constant Device Threads (grid) / CUDA Runtime API	Program/ <i>CUDA context</i>	Per device
<code>__managed__</code>	Host and Device (automatic)	Host/Device Threads	Program	Per program
<code>__shared__</code>	Device (streaming multiprocessor)	Block Threads	Block	Block
no specifier	Device (registers)	Single Thread	Single Thread	Single Thread

- `__device__`, `__tile__` and `__constant__` variables can be accessed from the host using the *CUDA Runtime API* functions `cudaGetSymbolAddress()`, `cudaGetSymbolSize()`, `cudaMemcpyToSymbol()`, and `cudaMemcpyFromSymbol()`.
- `__constant__` variables are read-only in device code and can only be modified from the host using the *CUDA Runtime API*.

The following example illustrates how to use these APIs:

```

__device__ float device_var = 4.0f; // Variable in device memory
__constant__ float constant_mem_var = 4.0f; // Variable in constant memory
// For readability, the following
// example focuses on a device variable.
int main() {
    float* device_ptr;
    cudaGetSymbolAddress((void**) &device_ptr, device_var); // Gets
    // address of device_var

    size_t symbol_size;
    cudaGetSymbolSize(&symbol_size, device_var); //
    // Retrieves the size of the symbol (4 bytes).

    float host_var;
    cudaMemcpyFromSymbol(&host_var, device_var, sizeof(host_var)); // Copies
    // from device to host.

    host_var = 3.0f;
    cudaMemcpyToSymbol(device_var, &host_var, sizeof(host_var)); // Copies
    // from host to device.
}

```

See the example on [Compiler Explorer](#).

5.4.1.2.1 `__shared__` Memory

`__shared__` memory variables can have a static size, which is determined at compile time, or a dynamic size, which is determined at kernel launch time. See the [Kernel Configuration](#) section for details on specifying the shared memory size at run time.

Shared memory constraints:

- ▶ Variables with a dynamic size must be declared as an external array or as a pointer.
- ▶ Variables with a static size cannot be initialized in their declaration.

The following example illustrates how to declare and size `__shared__` variables:

```
extern __shared__ char dynamic_smem_pointer[];
// extern __shared__ char* dynamic_smem_pointer; alternative syntax

__global__ void kernel() { // or a __device__ function
    __shared__ int smem_var1[4];           // static size
    auto smem_var2 = (int*) dynamic_smem_pointer; // dynamic size
}

int main() {
    size_t shared_memory_size = 16;
    kernel<<<1, 1, shared_memory_size>>>();
    cudaDeviceSynchronize();
}
```

See the example on [Compiler Explorer](#).

5.4.1.2.2 `__managed__` Memory

`__managed__` variables have the following restrictions:

- ▶ The address of a `__managed__` variable is not a constant expression.
- ▶ A `__managed__` variable shall not have a reference type `T&`.
- ▶ The address or value of a `__managed__` variable shall not be used when the CUDA runtime may not be in a valid state, including the following cases:
 - ▶ In static/dynamic initialization or destruction of an object with static or `thread_local` storage duration.
 - ▶ In code that executes after `exit()` has been called. For example, a function marked with `__attribute__((destructor))`.
 - ▶ In code that executes when the CUDA runtime may not be initialized. For example, a function marked with `__attribute__((constructor))`.
- ▶ A `__managed__` variable cannot be used as an unparenthesized id-expression argument to a `decltype()` expression.
- ▶ `__managed__` variables have the same coherence and consistency behavior as specified for [dynamically allocated managed memory](#).
- ▶ See also the restrictions for [local variables](#).

Here are examples of legal and illegal uses of `__managed__` variables:


```

#include <cassert>

__device__ __managed__ int global_var = 10; // OK

int* ptr = &global_var; // ERROR: use of a managed
    ↪ variable in static initialization

struct MyStruct1 {
    int field;
    MyStruct1() : field(global_var) {};
};

struct MyStruct2 {
    ~MyStruct2() { global_var = 10; }
};

MyStruct1 temp1; // ERROR: use of managed variable in dynamic initialization

MyStruct2 temp2; // ERROR: use of managed variable in the destructor of
    // object with static storage duration

__device__ __managed__ const int const_var = 10; // ERROR: const-
    ↪ qualified type

__device__ __managed__ int& reference = global_var; // ERROR: reference
    ↪ type

template <int* Addr>
struct MyStruct3 {};

MyStruct3<&global_var> temp; // ERROR: address of managed variable is not
    ↪ a constant expression

__global__ void kernel(int* ptr) {
    assert(ptr == &global_var); // OK
    global_var = 20; // OK
}

int main() {
    int* ptr = &global_var; // OK
    kernel<<<1, 1>>>(ptr);
    cudaDeviceSynchronize();
    global_var++; // OK
    decltype(global_var) var1; // ERROR: managed variable used as
    ↪ unparenthesized argument to decltype

    decltype((global_var)) var2; // OK
}

```

5.4.1.2.3 __tile__ Variables

The `__tile__` memory space specifier is similar to `__device__`. Variables marked with `__tile__` are allocated in device global memory and can be accessed by CUDA Runtime API functions. Unlike `__device__` variables, a `__tile__` variable is directly accessible from tile code.

In general, `__device__` variables may not be directly accessed by name from `__tile__` or `__tile_global__` code and `__tile__` variables may not be directly accessed by name from `__device__` or `__global__` code. However, both variables are allocated in device global memory and pointers to that memory may be accessed in either context.

The following example shows legal and illegal uses of tile variables:

```
__device__ int x_device = 0;
__tile__   int x_tile = 0;

// ERROR: Variable cannot be both device and tile
__device__ __tile__ int x_device_tile = 0;

__global__ void simt_function(int* in) {
    x_device = 1; // OK: __device__ variable accessed from SIMT code
    x_tile = 1;   // ERROR: __tile__ variable accessed from SIMT code

    *in = 1; // OK: May point to any device global memory, including x_tile.
}

__tile_global__ void tile_function(int* in) {
    x_device = 1; // ERROR: __device__ variable accessed from tile code
    x_tile = 1;   // OK: __tile__ variable accessed from tile code

    *in = 1; // OK: May point to any device global memory, including x_device.
}

int main() {
    int* x_device_ptr;
    cudaGetSymbolAddress((void**) &x_device_ptr, x_device); // Gets address
    ↪ of x_device

    int* x_tile_ptr;
    cudaGetSymbolAddress((void**) &x_tile_ptr, x_tile); // Gets address
    ↪ of x_tile

    // OK: Passing pointer to tile variable into SIMT kernel
    simt_function<<<1,1>>>(x_tile_ptr);
    cudaDeviceSynchronize();

    // OK: Passing pointer to device variable into tile kernel
    tile_function<<<1,1>>>(x_device_ptr);
    cudaDeviceSynchronize();
}
```

A `__tile__` variable may not contain a subobject of pointer or reference type. For example:

```
__tile__ int* ptr; // ERROR
```

(continues on next page)

(continued from previous page)

```
struct S1 { int* ptr; };
__tile__ S1 val; // ERROR
```

5.4.1.3 Inlining Specifiers

The following specifiers can be used to control inlining for `__host__` and `__device__` functions:

- ▶ `__noinline__`: Instructs nvcc not to inline the function.
- ▶ `__forceinline__`: Forces nvcc to inline the function within a single translation unit.
- ▶ `__inline_hint__`: Enables aggressive inlining across translation units when using *Link-Time Optimization*.

These specifiers are mutually exclusive. These specifiers are ignored when applied to `__tile__` functions.

5.4.1.4 `__restrict__` Pointers

nvcc supports restricted pointers via the `__restrict__` keyword.

Pointer aliasing occurs when two or more pointers refer to overlapping memory regions. This can inhibit optimizations such as code reordering and common sub-expression elimination.

A restrict-qualified pointer is a promise from the programmer that for the lifetime of the pointer, the memory it points to will only be accessed through that pointer. This allows the compiler to perform more aggressive optimizations.

- ▶ all threads that access the device function only read from it; or
- ▶ at most one thread writes to it, and no other thread reads from it.

The following example illustrates an aliasing issue and demonstrates how using a restricted pointer can help the compiler reduce the number of instructions:

```
__device__
void device_function(const float* a, const float* b, float* c) {
    c[0] = a[0] * b[0];
    c[1] = a[0] * b[0];
    c[2] = a[0] * b[0] * a[1];
    c[3] = a[0] * a[1];
    c[4] = a[0] * b[0];
    c[5] = b[0];
    ...
}
```

Because the pointers `a`, `b`, and `c` may be aliased, any write through `c` could modify elements of `a` or `b`. To guarantee functional correctness, the compiler cannot load `a[0]` and `b[0]` into registers, multiply them, and store the result in both `c[0]` and `c[1]`. This is because the results would differ from the abstract execution model if `a[0]` and `c[0]` were at the same location. The compiler cannot take advantage of the common sub-expression. Similarly, the compiler cannot reorder the computation of `c[4]` with the computations of `c[0]` and `c[1]` because a preceding write to `c[3]` could alter the inputs to the computation of `c[4]`.

By declaring `a`, `b`, and `c` as restricted pointers, the programmer informs the compiler that the pointers are not aliased. This means that writing to `c` will never overwrite the elements of `a` or `b`. This changes the function prototype as follows:

```
__device__
void device_function(const float* __restrict__ a, const float* __restrict__ b,
    ↪ float* __restrict__ c);
```

Note that all pointer arguments must be restricted for the compiler optimizer to be effective. With the addition of the `__restrict__` keywords, the compiler can reorder and perform common sub-expression elimination at will while maintaining identical functionality to the abstract execution model.

```
__device__
void device_function(const float* __restrict__ a, const float* __restrict__ b,
    ↪ float* __restrict__ c) {
    float t0 = a[0];
    float t1 = b[0];
    float t2 = t0 * t1;
    float t3 = a[1];
    c[0]      = t2;
    c[1]      = t2;
    c[4]      = t2;
    c[2]      = t2 * t3;
    c[3]      = t0 * t3;
    c[5]      = t1;
    ...
}
```

See the example on [Compiler Explorer](#).

The result is a reduced number of memory accesses and computations, balanced by an increase in register pressure from caching loads and common sub-expressions in registers.

Since register pressure is a critical issue in many CUDA codes, the use of restricted pointers can negatively impact performance by reducing occupancy.

Accesses to `__global__` function const pointers marked with `__restrict__` are compiled as read-only cache loads, similar to the PTX `ld.global.nc` or `__ldg()` *low-level load and store functions* instructions.

```
__global__
void kernel1(const float* in, float* out) {
    *out = *in; // PTX: ld.global
}

__global__
void kernel2(const float* __restrict__ in, float* out) {
    *out = *in; // PTX: ld.global.nc
}
```

See the example on [Compiler Explorer](#).

5.4.1.5 `__grid_constant__` Parameters

Annotating a `__global__` function parameter with `__grid_constant__` prevents the compiler from creating a per-thread copy of the parameter. Instead, all threads in the grid will access the parameter through a single address, which can improve performance.

The `__grid_constant__` parameter has the following properties:

- ▶ It has the lifetime of the kernel.
- ▶ It is private to a single kernel, meaning the object is not accessible to threads from other grids, including sub-grids.
- ▶ All threads in the kernel see the same address.
- ▶ It is read-only. Modifying a `__grid_constant__` object or any of its sub-objects, including mutable members, is undefined behavior.

Requirements:

- ▶ Kernel parameters annotated with `__grid_constant__` must have const-qualified non-reference types.
- ▶ All function declarations must be consistent with any `__grid_constant__` parameters.
- ▶ Function template specializations must match the primary template declaration with respect to any `__grid_constant__` parameters.
- ▶ Function template instantiations must also match the primary template declaration with respect to any `__grid_constant__` parameters.
- ▶ The `__grid_constant__` annotation is ignored in `__tile_global__` function parameters.

Examples:

```
struct MyStruct {
    int x;
    mutable int y;
};

__device__ void external_function(const MyStruct&);

__global__ void kernel(const __grid_constant__ MyStruct s) {
    // s.x++; // Compile error: tried to modify read-only memory
    // s.y++; // Undefined Behavior: tried to modify read-only memory

    // Compiler will NOT create a per-thread local copy of "s":
    external_function(s);
}
```

See the example on [Compiler Explorer](#).

5.4.1.6 Annotation Summary

The following table summarizes the CUDA annotations and reports which execution space each annotation applies to and where it is valid.

Table 41: Annotation Summary

Annotation	<code>__host__</code> / <code>__device__</code> / <code>__host__ __device__</code>	<code>__global__</code>
<code>__noinline__</code> , <code>__forceinline__</code> , <code>__inline_hint__</code>	Function	×
<code>__restrict__</code>	Pointer Parameter	Pointer Parameter
<code>__grid_constant__</code>	×	Parameter
<code>__launch_bounds__</code>	×	Function
<code>__maxnreg__</code>	×	Function
<code>__cluster_dims__</code>	×	Function

5.4.2. Built-in Types and Variables

5.4.2.1 Host Compiler Type Extensions

The use of non-standard arithmetic types is permitted by CUDA, as long as the host compiler supports it. The following types are supported:

- ▶ 128-bit integer type `__int128`.
 - ▶ Supported on Linux when the host compiler defines the `__SIZEOF_INT128__` macro.
- ▶ 128-bit floating-point types `__float128` and `_Float128` are available on GPU devices with compute capability 10.0 and later. A constant expression of `__float128` type may be processed by the compiler in a floating-point representation with lower precision.
 - ▶ Supported on Linux x86 when the host compiler defines the `__SIZEOF_FLOAT128__` or `__FLOAT128__` macros.
- ▶ `_Complex` types are only supported in host code.

The 128-bit integer and floating point types are not supported in tile code.

5.4.2.2 Built-in Variables

The values used to specify and retrieve the kernel configuration for the grid and blocks along the x, y, and z dimensions are of type `dim3`. The variables used to obtain the block and thread indices are of type `uint3`. Both `dim3` and `uint3` are trivial structures consisting of three unsigned values named x, y, and z. In C++11 and later, the default value of all components of `dim3` is 1.

Built-in device-only variables:

- ▶ `dim3 gridDim`: contains the dimensions of the grid, namely the number of thread blocks, along the x, y, and z dimensions.
- ▶ `dim3 blockDim`: contains the dimensions of the thread block, namely the number of threads, along the x, y, and z dimensions.
- ▶ `uint3 blockIdx`: contains the block index within the grid, along the x, y, and z dimensions.
- ▶ `uint3 threadIdx`: contains the thread index within the block, along the x, y, and z dimensions.
- ▶ `int warpSize`: A run-time value defined as the number of threads in a warp, commonly 32. See also *Warps and SIMT* for the definition of a warp.

These variables are not supported in tile code. To retrieve the block ID or grid size in tile code, use the `cuda::tiles::bid()` and `cuda::tiles::num_blocks()` APIs.

5.4.2.3 Built-in Types

CUDA provides vector types derived from basic integer and floating-point types that are supported for both the host and the device. The following table shows the available vector types.

Table 42: Vector Types

C++ Type	Fundamental	Vector X1	Vector X2	Vector X3	Vector X4
signed char		char1	char2	char3	char4
unsigned char		uchar1	uchar2	uchar3	uchar4
signed short		short1	short2	short3	short4
unsigned short		ushort1	ushort2	ushort3	ushort4
signed int		int1	int2	int3	int4
unsigned		uint1	uint2	uint3	uint4
signed long		long1	long2	long3	long4_16a/ long4_32a
unsigned long		ulong1	ulong2	ulong3	ulong4_16a/ ulong4_32a
signed long long		longlong1	longlong2	longlong3	longlong4_16a/ longlong4_32a
unsigned long	long	ulonglong1	ulonglong2	ulonglong3	ulonglong4_16a/ ulonglong4_32a
float		float1	float2	float3	float4
double		double1	double2	double3	double4_16a/ double4_32a

Note that `long4`, `ulong4`, `longlong4`, `ulonglong4`, and `double4` have been deprecated in CUDA 13, and may be removed in a future release.

The following table details the byte size and alignment requirements of the vector types:

Table 43: Alignment Requirements

Type	Size	Alignment
char1, uchar1	1	1
char2, uchar2	2	2
char3, uchar3	3	1

continues on next page

Table 43 – continued from previous page

Type	Size	Alignment
char4, uchar4	4	4
short1, ushort1	2	2
short2, ushort2	4	4
short3, ushort3	6	2
short4, ushort4	8	8
int1, uint1	4	4
int2, uint2	8	8
int3, uint3	12	4
int4, uint4	16	16
long1, ulong1	4/8 *	4/8 *
long2, ulong2	8/16 *	8/16 *
long3, ulong3	12/24 *	4/8 *
long4, ulong4 (deprecated)	16/32 *	16 *
long4_16a, ulong4_16a	16/32 *	16
long4_32a, ulong4_32a	16/32 *	32
longlong1, ulonglong1	8	8
longlong2, ulonglong2	16	16
longlong3, ulonglong3	24	8
longlong4, ulonglong4 (deprecated)	32	16
longlong4_16a, ulonglong4_16a	32	16
longlong4_32a, ulonglong4_32a	32	32
float1	4	4
float2	8	8
float3	12	4
float4	16	16
double1	8	8
double2	16	16
double3	24	8
double4 (deprecated)	32	16
double4_16a	32	16
double4_32a	32	32

* long is 4 bytes on C++ LLP64 data model (Windows 64-bit), while it is 8 bytes on C++ LP64 data model (Linux 64-bit).

Vector types are structures. Their first, second, third, and fourth components are accessible through the `x`, `y`, `z`, and `w` fields, respectively.

```
int sum(int4 value) {
    return value.x + value.y + value.z + value.w;
}
```

They all have a factory function of the form `make_<type_name>()`; for example:

```
int4 add_one(int x, int y, int z, int w) {
    return make_int4(x + 1, y + 1, z + 1, w + 1);
}
```

If host code is not compiled with `nvcc`, the vector types and related functions can be imported by including the `cuda_runtime.h` header provided in the CUDA toolkit.

5.4.3. Kernel Configuration

Any call to a `__global__` or `__tile_global__` function must specify an *execution configuration* for that call. This execution configuration defines the dimensions of the grid and blocks that will be used to execute the function on the device, as well as the associated *stream*.

The execution configuration is specified by inserting an expression in the form `<<<grid_dim, block_dim, dynamic_smem_bytes, stream>>>` between the function name and the parenthesized argument list, where:

- ▶ `grid_dim` is of type `dim3` and specifies the dimension and size of the grid, such that `grid_dim.x * grid_dim.y * grid_dim.z` equals the number of blocks being launched.
- ▶ `block_dim` is of type `dim3` and specifies the dimension and size of each block, such that `block_dim.x * block_dim.y * block_dim.z` equals the number of threads per block. For `__tile_global__` kernels, the `block_dim` must be 1 because the compiler will decide how many threads to schedule when launching the tile kernel.
- ▶ `dynamic_smem_bytes` is an optional `size_t` argument that defaults to zero. It specifies the number of bytes in shared memory that are dynamically allocated per block for this call in addition to the statically allocated memory. This memory is used by extern `__shared__` arrays (see [__shared__ Memory](#)).
- ▶ `stream` is of type `cudaStream_t` (pointer) and specifies the associated stream. `stream` is an optional argument that defaults to `NULL`.

The following example shows a kernel function declaration and call:

```
__global__ void kernel(float* parameter);

kernel<<<grid_dim, block_dim, dynamic_smem_bytes>>>(parameter);
```

The arguments for the execution configuration are evaluated before the arguments for the actual function.

The function call fails if `grid_dim` or `block_dim` exceeds the maximum sizes allowed for the device, as specified in [Compute Capabilities](#), or if `dynamic_smem_bytes` is greater than the available shared memory after accounting for statically allocated memory.

5.4.3.1 Thread Block Cluster

Compute capability 9.0 and higher allow users to specify compile-time thread block cluster dimensions so that the kernels can use the *cluster hierarchy* in CUDA. The compile-time cluster dimension can be specified using the `__cluster_dims__` attribute with the following syntax: `__cluster_dims__((x, [y, [z]]))`. The example below shows a compile-time cluster size of 2 in the X dimension and 1 in the Y and Z dimensions.

```
__global__ void __cluster_dims__(2, 1, 1) kernel(float* parameter);
```

The default form of `__cluster_dims__()` specifies that a kernel is to be launched as a grid cluster. If a cluster dimension is not specified, the user can specify it at launch time. Failing to specify a dimension at launch time will result in a launch-time error. The `__cluster_dims__` attribute may not be specified on `__tile_global__` kernels because the compiler decides how to schedule tile kernels across clusters.

The dimensions of the thread block cluster can also be specified at runtime, and the kernel with the cluster can be launched using the `cudaLaunchKernelEx` API. This API takes a configuration argument of type `cudaLaunchConfig_t`, a kernel function pointer, and kernel arguments. The example below shows runtime kernel configuration.

```
__global__ void kernel(float parameter1, int parameter2) {}

int main() {
    cudaLaunchConfig_t config = {0};
    // The grid dimension is not affected by cluster launch, and is still
    ↪ enumerated
    // using the number of blocks.
    // The grid dimension should be a multiple of cluster size.
    config.gridDim      = dim3{4}; // 4 blocks
    config.blockDim     = dim3{32}; // 32 threads per block
    config.dynamicSmemBytes = 1024; // 1 KB

    cudaLaunchAttribute attribute[1];
    attribute[0].id      = cudaLaunchAttributeClusterDimension;
    attribute[0].val.clusterDim.x = 2; // Cluster size in X-dimension
    attribute[0].val.clusterDim.y = 1;
    attribute[0].val.clusterDim.z = 1;
    config.attrs         = attribute;
    config.numAttrs      = 1;

    float parameter1 = 3.0f;
    int   parameter2 = 4;
    cudaLaunchKernelEx(&config, kernel, parameter1, parameter2);
}
```

See the example on [Compiler Explorer](#).

5.4.3.2 Launch Bounds

As discussed in the *Kernel Launch and Occupancy* section, using fewer registers allows more threads and thread blocks to reside on a multiprocessor, which improves performance.

Therefore, the compiler uses heuristics to minimize register usage while keeping *register spilling* and instruction count to a minimum. Applications can optionally aid these heuristics by providing

additional information to the compiler in the form of launch bounds that are specified using the `__launch_bounds__()` qualifier in the definition of a `__global__` function:

```
__global__ void
__launch_bounds__(maxThreadsPerBlock, minBlocksPerMultiprocessor,
    ↪maxBlocksPerCluster)
MyKernel(...) {
    ...
}
```

- ▶ `maxThreadsPerBlock` specifies the maximum number of threads per block with which the application will ever launch `MyKernel()`; it compiles to the `.maxntid` PTX directive.
- ▶ `minBlocksPerMultiprocessor` is optional and specifies the desired minimum number of resident blocks per multiprocessor; it compiles to the `.minntapersm` PTX directive.
- ▶ `maxBlocksPerCluster` is optional and specifies the desired maximum number of thread blocks per cluster with which the application will ever launch `MyKernel()`; it compiles to the `.maxclusterrank` PTX directive.

If launch bounds are specified, the compiler first derives the upper limit, *L*, on the number of registers that the kernel should use. This ensures that `minBlocksPerMultiprocessor` blocks (or a single block, if `minBlocksPerMultiprocessor` is not specified) of `maxThreadsPerBlock` threads can reside on the multiprocessor. See the [occupancy](#) section for the relationship between the number of registers used by a kernel and the number of registers allocated per block. The compiler then optimizes register usage as follows:

- ▶ If the initial register usage exceeds *L*, the compiler reduces it until it is less than or equal to *L*. This usually results in increased local memory usage and/or a higher number of instructions.
- ▶ If the initial register usage is lower than *L*
 - ▶ If `maxThreadsPerBlock` is specified but `minBlocksPerMultiprocessor` is not, the compiler uses `maxThreadsPerBlock` to determine the register usage thresholds for the transitions between *n* and *n* + 1 resident blocks. This occurs when using one less register makes room for an additional resident block. Then, the compiler applies similar heuristics as when no launch bounds are specified.
 - ▶ If both `minBlocksPerMultiprocessor` and `maxThreadsPerBlock` are specified, the compiler may increase register usage up to *L* in order to reduce the number of instructions and better hide the latency of single-threaded instructions.

A kernel will fail to launch if it is executed with:

- ▶ more threads per block than its launch bound `maxThreadsPerBlock`.
- ▶ more thread blocks per cluster than its launch bound `maxBlocksPerCluster`.

The per-thread resources required by a CUDA kernel may limit the maximum block size in an undesirable way. To maintain forward compatibility with future hardware and toolkits, and to ensure that at least one thread block can run on a streaming multiprocessor, developers should include the single argument `__launch_bounds__(maxThreadsPerBlock)` which specifies the largest block size with which the kernel will launch. Failure to do so could result in “too many resources requested for launch” errors. Providing the two-argument version of `__launch_bounds__(maxThreadsPerBlock, minBlocksPerMultiprocessor)` can improve performance in some cases. The best value for `minBlocksPerMultiprocessor` should be determined through a detailed analysis of each kernel.

The optimal launch bounds for a kernel typically differ across major architecture revisions. The following code sample illustrates how this is managed in device code with the `__CUDA_ARCH__` [macro](#).

```

#define THREADS_PER_BLOCK 256

#if __CUDA_ARCH__ >= 900
    #define MY_KERNEL_MAX_THREADS (2 * THREADS_PER_BLOCK)
    #define MY_KERNEL_MIN_BLOCKS 3
#else
    #define MY_KERNEL_MAX_THREADS THREADS_PER_BLOCK
    #define MY_KERNEL_MIN_BLOCKS 2
#endif

__global__ void
__launch_bounds__(MY_KERNEL_MAX_THREADS, MY_KERNEL_MIN_BLOCKS)
MyKernel(...) {
    ...
}

```

When `MyKernel` is invoked with the maximum number of threads per block, which is specified as the first parameter of `__launch_bounds__()`, it is tempting to use `MY_KERNEL_MAX_THREADS` as the number of threads per block in the execution configuration:

```

// Host code
MyKernel<<<blocksPerGrid, MY_KERNEL_MAX_THREADS>>>(...);

```

However, this will not work, since `__CUDA_ARCH__` is undefined in host code as mentioned in the [Execution Space Specifiers](#) section. Therefore, `MyKernel` will launch with 256 threads per block. The number of threads per block should instead be determined:

- Either at compile time using a macro or constant that does not depend on `__CUDA_ARCH__`, for example

```

// Host code
MyKernel<<<blocksPerGrid, THREADS_PER_BLOCK>>>(...);

```

- Or at runtime based on the compute capability

```

// Host code
cudaGetDeviceProperties(&deviceProp, device);
int threadsPerBlock = (deviceProp.major >= 9) ? 2 * THREADS_PER_BLOCK :
    → THREADS_PER_BLOCK;
MyKernel<<<blocksPerGrid, threadsPerBlock>>>(...);

```

The `--resource-usage` compiler option reports register usage. The [CUDA profiler](#) reports occupancy, which can be used to derive the number of resident blocks.

5.4.3.3 Maximum Number of Registers per Thread

To enable low-level performance tuning, CUDA C++ offers the `__maxnreg__()` function qualifier, which passes performance tuning information to the backend optimizing compiler. The `__maxnreg__()` qualifier specifies the maximum number of registers that can be allocated to a single thread in a thread block. In the definition of a `__global__` function:

```

__global__ void
__maxnreg__(maxNumberRegistersPerThread)
MyKernel(...) {

```

(continues on next page)

(continued from previous page)

```
...
}
```

The `maxNumberRegistersPerThread` variable specifies the maximum number of registers to be allocated to a single thread in a thread block of the kernel `MyKernel()`; it compiles to the `.maxnreg` PTX directive.

The `__launch_bounds__()` and `__maxnreg__()` qualifiers cannot be applied to the same kernel together.

The `--maxrregcount <N>` compiler option can be used to control register usage for all `__global__` functions in a file. This option is ignored for kernel functions with the `__maxnreg__` qualifier.

5.4.4. Synchronization Primitives

5.4.4.1 Thread Block Synchronization Functions

```
void __syncthreads();
int __syncthreads_count(int predicate);
int __syncthreads_and(int predicate);
int __syncthreads_or(int predicate);
```

The intrinsics coordinate communication among threads within the same block. When threads in a block access the same addresses in shared or global memory, read-after-write, write-after-read, or write-after-write hazards can occur. These hazards can be avoided by synchronizing threads between such accesses.

The intrinsics have the following semantics:

- `__syncthreads*()` wait until all non-exited threads in the thread block simultaneously reach the same `__syncthreads*()` intrinsic call in the program or exit.
- `__syncthreads*()` provide memory ordering among participating threads: the call to `__syncthreads*()` intrinsics strongly happens before (see [C++ specification \[intro.races\]](#)) any participating thread is unblocked from the wait or exits.

The following example shows how to use `__syncthreads()` to synchronize threads within a thread block and safely sum the elements of an array shared among the threads:

```
#include <cuda_runtime_api.h>
#include <memory.h>
#include <cstdlib>
#include <ctime>
#include <stdio.h>

// assuming blockDim.x is 128
__global__ void example_syncthreads(int* input_data, int* output_data)
{
    __shared__ int shared_data[128];
    shared_data[threadIdx.x] = input_data[blockDim.x*blockIdx.x + threadIdx.
↪x];

    // All threads synchronize, guaranteeing all writes to 'shared_data' are
```

(continues on next page)

(continued from previous page)

```

→ordered
// before any thread is unblocked from '__syncthreads()':
__syncthreads();

// A single thread safely reads 'shared_data':
if (threadIdx.x == 0) {
    float sum = 0;
    for (int i = 0; i < blockDim.x; ++i) {
        sum += shared_data[i];
    }
    output_data[blockIdx.x] = sum;
}
}

void initArray(int* A, int length)
{
    std::srand(std::time({}));
    for(int i=0; i<length; i++)
    {
        A[i] = int(10.f * (rand() / (float)RAND_MAX));
    }
}

int main(int argc, char** arg)
{
    constexpr int block_size = 128;
    constexpr int input_length = 1024;
    constexpr int output_length = input_length / block_size;

    // Pointers to memory vectors
    int* input = nullptr;
    int* output = nullptr;
    int* comparisonResult = (int*)malloc(output_length*sizeof(int));

    // Use unified memory to allocate buffers
    cudaMallocManaged(&input, input_length*sizeof(int));
    cudaMallocManaged(&output, output_length*sizeof(int));

    initArray(input, input_length);
    int grid_size = input_length / block_size;

    example_syncthreads<<<grid_size, block_size>>>(input, output);
    cudaDeviceSynchronize();

    for(int i=0; i<output_length; i++)
    {
        comparisonResult[i] = 0;
        for(int j=0; j < block_size; j++)
        {
            comparisonResult[i] += input[i*block_size + j];
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
}

for(int i=0; i< output_length; i++)
{
    if(output[i] != comparisonResult[i])
    {
        printf("Results do not match at index %d: %d != %d\n", i,
→output[i], comparisonResult[i]);
        exit(-1);
    }
}
printf("Test passed\n");

return 0;
}

```

The `__syncthreads*()` intrinsics are permitted in conditional code, but only if the condition evaluates uniformly across the entire thread block. Otherwise, execution may hang or produce unintended side effects.

The following example demonstrates a valid behavior:

```

// assuming blockDim.x is 128
__global__ void syncthreads_valid_behavior(int* input_data, int* output_data)
→{
    __shared__ int shared_data[128];
    shared_data[threadIdx.x] = input_data[threadIdx.x];
    if (blockIdx.x > 0) { // CORRECT, uniform condition across all block
→threads
        __syncthreads();
        output_data[threadIdx.x] = shared_data[128 - threadIdx.x];
    }
}

```

while the following examples exhibit invalid behavior, such as kernel hang, or undefined behavior:

```

// assuming blockDim.x is 128
__global__ void syncthreads_invalid_behavior1(int* input_data, int* output_
→data) {
    __shared__ int shared_data[256];
    shared_data[threadIdx.x] = input_data[threadIdx.x];
    if (threadIdx.x > 0) { // WRONG, non-uniform condition
        __syncthreads(); // Undefined Behavior
        output_data[threadIdx.x] = shared_data[128 - threadIdx.x];
    }
}

```

```

// assuming blockDim.x is 128
__global__ void syncthreads_invalid_behavior2(int* input_data, int* output_
→data) {
    __shared__ int shared_data[256];
    shared_data[threadIdx.x] = input_data[threadIdx.x];

```

(continues on next page)

(continued from previous page)

```

for (int i = 0; i < blockDim.x; ++i) {
    if (i == threadIdx.x) { // WRONG, non-uniform condition
        __syncthreads();    // Undefined Behavior
    }
}
output_data[threadIdx.x] = shared_data[128 - threadIdx.x];
}

```

`__syncthreads()` variants with predicate:

```
int __syncthreads_count(int predicate);
```

is identical to `__syncthreads()` except that it evaluates a predicate for all non-exited threads in the block and returns the number of threads for which the predicate evaluates to a non-zero value.

```
int __syncthreads_and(int predicate);
```

is identical to `__syncthreads()` except that it evaluates the predicate for all non-exited threads in the block. It returns a non-zero value if and only if the predicate evaluates to a non-zero value for all of them.

```
int __syncthreads_or(int predicate);
```

is identical to `__syncthreads()` except that it evaluates the predicate for all non-exited threads in the block. It returns a non-zero value if and only if the predicate evaluates to a non-zero value one or more of them.

5.4.4.2 Warp Synchronization Function

```
void __syncwarp(unsigned mask = 0xFFFFFFFF);
```

The intrinsic function `__syncwarp()` coordinates communication between the threads within the same warp. When some threads within a warp access the same addresses in shared or global memory, potential read-after-write, write-after-read, or write-after-write hazards may occur. These data hazards can be avoided by synchronizing the threads between these accesses.

Calling `__syncwarp(mask)` provides memory ordering among the participating threads within a warp named in `mask`: the call to `__syncwarp(mask)` strongly happens before (see [C++ specification \[intro.races\]](#)) any warp thread named in `mask` is unblocked from the wait or exits.

The functions are subject to the [Warp __sync Intrinsic Constraints](#).

The following example demonstrates how to use `__syncwarp()` to synchronize threads within a warp to safely access a shared memory array:

```

__global__ void example_syncwarp(int* input_data, int* output_data) {
    if (threadIdx.x < warpSize) {
        __shared__ int shared_data[warpSize];
        shared_data[threadIdx.x] = input_data[threadIdx.x];

        __syncwarp(); // equivalent to __syncwarp(0xFFFFFFFF)
        if (threadIdx.x == 0)
            output_data[0] = shared_data[1];
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
}

```

5.4.4.3 Memory Fence Functions

The CUDA programming model assumes a weakly ordered memory model. In other words, the order in which a CUDA thread writes data to shared memory, global memory, page-locked host memory, or the memory of a peer device is not necessarily the order in which another CUDA or host thread observes the data being written. Reading from or writing to the same memory location without memory fences or synchronization results in undefined behavior.

In the following example, thread 1 executes `writeXY()`, while thread 2 executes `readXY()`.

```

__device__ int X = 1, Y = 2;

__device__ void writeXY() {
    X = 10;
    Y = 20;
}

__device__ void readXY() {
    int B = Y;
    int A = X;
}

```

The two threads simultaneously read and write to the same memory locations, X and Y. Any data race results in undefined behavior and has no defined semantics. Therefore, the resulting values for A and B can be anything.

Memory fence and synchronization functions enforce a [sequentially consistent ordering](#) of memory accesses. These functions differ in the [thread scope](#) in which orderings are enforced, but are independent of the accessed memory space, including shared memory, global memory, page-locked host memory, and the memory of a peer device.

Hint

It is suggested to use `cuda::atomic_thread_fence` provided by `libcu++` whenever possible for safety and portability reasons.

Block-level memory fence

CUDA C++

```

// <cuda/atomic> header
cuda::atomic_thread_fence(cuda::memory_order_seq_cst, cuda::thread_scope_
    ↪ block);

```

ensures that:

- All writes to all memory made by the calling thread before the call to `cuda::atomic_thread_fence()` are observed by all threads in the calling thread's block as occurring before all writes to all memory made by the calling thread after the call to `cuda::atomic_thread_fence()`;

- All reads from all memory made by the calling thread before the call to `cuda::atomic_thread_fence()` are ordered before all reads from all memory made by the calling thread after the call to `cuda::atomic_thread_fence()`.

Intrinsics

```
void __threadfence_block();
```

ensures that:

- All writes to all memory made by the calling thread before the call to `__threadfence_block()` are observed by all threads in the calling thread's block as occurring before all writes to all memory made by the calling thread after the call to `__threadfence_block()`;
- All reads from all memory made by the calling thread before the call to `__threadfence_block()` are ordered before all reads from all memory made by the calling thread after the call to `__threadfence_block()`.

Device-level memory fence

CUDA C++

```
cuda::atomic_thread_fence(cuda::memory_order_seq_cst, cuda::thread_scope_  
↪ device);
```

ensures that:

- No writes to all memory made by the calling thread after the call to `cuda::atomic_thread_fence()` are observed by any thread in the device as occurring before any write to all memory made by the calling thread before the call to `cuda::atomic_thread_fence()`.

Intrinsics

```
void __threadfence();
```

ensures that:

- No writes to all memory made by the calling thread after the call to `__threadfence()` are observed by any thread in the device as occurring before any write to all memory made by the calling thread before the call to `__threadfence()`.

System-level memory fence

CUDA C++

```
cuda::atomic_thread_fence(cuda::memory_order_seq_cst, cuda::thread_scope_  
↪ system);
```

ensures that:

- All writes to all memory made by the calling thread before the call to `cuda::atomic_thread_fence()` are observed by all threads in the device, host threads, and all threads in peer devices as occurring before all writes to all memory made by the calling thread after the call to `cuda::atomic_thread_fence()`.

Intrinsics

```
void __threadfence_system();
```

ensures that:

- All writes to all memory made by the calling thread before the call to `__threadfence_system()` are observed by all threads in the device, host threads, and all threads in peer devices as occurring before all writes to all memory made by the calling thread after the call to `__threadfence_system()`.

In the previous code sample, we can insert memory fences in the code as follows:

CUDA C++

```
#include <cuda/atomic>

__device__ int X = 1, Y = 2;

__device__ void writeXY() {
    X = 10;
    cuda::atomic_thread_fence(cuda::memory_order_seq_cst, cuda::thread_scope_
    ↪device);
    Y = 20;
}

__device__ void readXY() {
    int B = Y;
    cuda::atomic_thread_fence(cuda::memory_order_seq_cst, cuda::thread_scope_
    ↪device);
    int A = X;
}
```

Intrinsics

```
__device__ int X = 1, Y = 2;

__device__ void writeXY() {
    X = 10;
    __threadfence();
    Y = 20;
}

__device__ void readXY() {
    int B = Y;
    __threadfence();
    int A = X;
}
```

For this code, the following outcomes can be observed:

- A equal to 1 and B equal to 2, namely `readXY()` is executed before `writeXY()`,
- A equal to 10 and B equal to 20, namely `writeXY()` is executed before `readXY()`.
- A equal to 10 and B equal to 2.

- The case where A is 1 and B is 20 is not possible, as the memory fence ensures that the write to X is visible before the write to Y.

If threads 1 and 2 belong to the same block, it is enough to use a block-level fence. If threads 1 and 2 do not belong to the same block, a device-level fence must be used if they are CUDA threads from the same device, and a system-level fence must be used if they are CUDA threads from two different devices.

A common use case is illustrated by the following code sample, where threads consume data produced by other threads. This kernel computes the sum of an array of N numbers in a single call.

- Each block first sums a subset of the array and stores the result in global memory.
- When all the blocks have finished, the last block reads each of these partial sums from global memory and adds them together to obtain the final result.
- To determine which block finished last, each block atomically increments a counter to signal completion of computing and storing its partial sum (see the [Atomic Functions](#) section for further details). The last block receives a counter value equal to `gridDim.x - 1`.

Without a fence between storing the partial sum and incrementing the counter, the counter may increment before the partial sum is stored. This could cause the counter to reach `gridDim.x - 1` and allow the last block to start reading partial sums before they are updated in memory.

Note

The memory fence only affects the order in which memory operations are executed; it does not guarantee visibility of these operations to other threads.

In the code sample below, the visibility of the memory operations on the `result` variable is ensured by declaring it as `volatile`. For more details, see the [volatile-qualified variables](#) section.

```
#include <cuda/atomic>

__device__ int count = 0;

__global__ void sum(const float* array,
                   int N,
                   volatile float* result) {
    __shared__ bool isLastBlockDone;
    // Each block sums a subset of the input array.
    float partialSum = calculatePartialSum(array, N);

    if (threadIdx.x == 0) {
        // Thread 0 of each block stores the partial sum to global memory.
        // The compiler will use a store operation that bypasses the L1 cache
        // since the "result" variable is declared as volatile.
        // This ensures that the threads of the last block will read the
        ↪correct
        // partial sums computed by all other blocks.
        result[blockIdx.x] = partialSum;

        // Thread 0 makes sure that the increment of the "count" variable is
        // only performed after the partial sum has been written to global
        ↪memory.
    }
}
```

(continues on next page)

(continued from previous page)

```

    cuda::atomic_thread_fence(cuda::memory_order_seq_cst, cuda::thread_
→scope_device);

    // Thread 0 signals that it is done.
    int count_old = atomicInc(&count, gridDim.x);

    // Thread 0 determines if its block is the last block to be done.
    isLastBlockDone = (count_old == (gridDim.x - 1));
}
// Synchronize to make sure that each thread reads the correct value of
// isLastBlockDone.
__syncthreads();

if (isLastBlockDone) {
    // The last block sums the partial sums stored in result[0 .. gridDim.
→x-1]
    float totalSum = calculateTotalSum(result);

    if (threadIdx.x == 0) {
        // Thread 0 of last block stores the total sum to global memory and
        // resets the count variable, so that the next kernel call works
        // properly.
        result[0] = totalSum;
        count      = 0;
    }
}
}
}

```

5.4.5. Atomic Functions

Atomic functions perform read-modify-write operations on shared data, making them appear to execute in a single step. Atomicity ensures that each operation either completes fully or not at all, providing all participating threads with a consistent view of the data.

CUDA provides atomic functions in five ways:

Extended CUDA C++ atomic functions, `cuda::atomic` and `cuda::atomic_ref`.

- ▶ They are allowed in both host and device code.
- ▶ They follow the C++ standard atomic operations semantics.
- ▶ They allow specifying the thread scope of the atomic operations.

Standard C++ atomic functions, `cuda::std::atomic` and `cuda::std::atomic_ref`.

- ▶ They are allowed in both host and device code.
- ▶ They follow the C++ standard atomic operations semantics.
- ▶ They do not allow specifying the thread scope of the atomic operations.

Compiler built-in atomic functions, `__nv_atomic_<op>()`.

- ▶ They have been available since CUDA 12.8.
- ▶ They are only allowed in device code.
- ▶ They follow the C++ standard atomic memory order semantics.
- ▶ They allow specifying the thread scope of the atomic operations.
- ▶ They have the same memory ordering semantics as C++ standard atomic operations.
- ▶ They support a subset of the data types allowed by `cuda::std::atomic` and `cuda::std::atomic_ref`, except for 128-bit data types.
- ▶ They are not supported in tile code.

CUDA Tile C++ atomic functions (for example, `cuda::tiles::atomic_load`):

- ▶ They are allowed only in tile code.
- ▶ They follow the C++ standard atomic memory order semantics.
- ▶ They allow specifying a thread scope through `cuda::tiles::thread_scope`.

Legacy atomic functions, `atomic<Op>()`.

- ▶ They are only allowed in device code.
- ▶ They only support `memory_order_relaxed` C++ atomic memory semantics.
- ▶ They allow specifying the thread scope of the atomic operations as part of the function name.
- ▶ Unlike *built-in atomic functions*, legacy atomic functions only ensure atomicity and do not introduce synchronization points (fences).
- ▶ They support a subset of the data types allowed by *built-in atomic functions*. The atomic add operation supports additional data types.

Hint

Using the *Extended CUDA C++ atomic functions* provided by `libcudac++` is recommended for efficiency, safety, and portability.

5.4.5.1 Legacy Atomic Functions

Legacy atomic functions perform atomic read-modify-write operations on a 32-, 64-, or 128-bit word stored in global or shared memory. For example, the `atomicAdd()` function reads a word at a specific address in global or shared memory, adds a number to it, and writes the result back to the same address.

- ▶ Atomic functions can only be used in device functions.
- ▶ For vector types such as `__half2`, `__nv_bfloat162`, `float2`, and `float4`, the read-modify-write operation is performed on each element of the vector. The entire vector is not guaranteed to be atomic in a single access.

The atomic functions described in this section have a `memory_order_relaxed` and are only atomic at a particular thread scope:

- ▶ Atomic APIs without a suffix, for example `atomicAdd`, are atomic at scope `cuda::thread_scope_device`.

- ▶ Atomic APIs with the `_block` suffix, for example, `atomicAdd_block`, are atomic at scope `cuda::thread_scope_block`.
- ▶ Atomic APIs with the `_system` suffix, for example, `atomicAdd_system`, are atomic at scope `cuda::thread_scope_system` if they meet particular [conditions](#).

The following example shows the CPU and GPU atomically updating an integer value at address `addr`:

```
#include <cuda_runtime.h>

__global__ void atomicAdd_kernel(int* addr) {
    atomicAdd_system(addr, 10);
}

void test_atomicAdd(int device_id) {
    int* addr;
    cudaMallocManaged(&addr, 4);
    *addr = 0;

    cudaDeviceProp deviceProp;
    cudaGetDeviceProperties(&deviceProp, device_id);
    if (deviceProp.concurrentManagedAccess != 1) {
        return; // the device does not coherently access managed memory
        → concurrently with the CPU
    }

    atomicAdd_kernel<<<...>>>(addr);
    __sync_fetch_and_add(addr, 10); // CPU atomic operation
}
```

Note that any atomic operation can be implemented based on `atomicCAS()` (Compare and Swap). For example, `atomicAdd()` for single-precision floating-point numbers can be implemented as follows:

```
#include <cuda/memory>
#include <cuda/std/bit>

__device__ float customAtomicAdd(float* d_ptr, float value) {
    volatile unsigned* d_ptr_unsigned = reinterpret_cast<unsigned*>(d_ptr);
    unsigned old_value = *d_ptr_unsigned;
    unsigned assumed;
    do {
        assumed = old_value;
        float assumed_float = cuda::std::bit_cast<float>
        → (assumed);
        float expected_value = assumed_float + value;
        unsigned expected_value_unsigned = cuda::std::bit_cast<unsigned>
        → (expected_value);
        old_value = atomicCAS(d_ptr_unsigned, assumed,
        → expected_value_unsigned);
        // Note: uses integer comparison to avoid hang in case of NaN (since NaN !
        → = NaN)
    } while (assumed != old_value);
    return cuda::std::bit_cast<float>(old_value);
}
```

See the example on [Compiler Explorer](#).

5.4.5.1.1 `atomicAdd()`

```
T atomicAdd(T* address, T val);
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes `old + val`.
3. Stores the result back to memory at the same address.

The function returns the old value.

`atomicAdd()` supports the following data types:

- ▶ `int`, `unsigned`, `unsigned long long`, `float`, `double`, `__half2`, `__half`.
- ▶ `__nv_bfloat16`, `__nv_bfloat162` on devices of compute capability 8.x and higher.
- ▶ `float2`, `float4` on devices of compute capability 9.x and higher, and only supported for global memory addresses.

The atomicity of `atomicAdd()` applied to vector types, for example `__half2` or `float4`, is guaranteed separately for each of the components; the entire vector is not guaranteed to be atomic as a single access.

5.4.5.1.2 `atomicSub()`

```
T atomicSub(T* address, T val);
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes `old - val`.
3. Stores the result back to memory at the same address.

The function returns the old value.

`atomicSub()` supports the following data types:

- ▶ `int`, `unsigned`

5.4.5.1.3 `atomicInc()`

```
unsigned atomicInc(unsigned* address, unsigned val);
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes `old >= val ? 0 : (old + 1)`.
3. Stores the result back to memory at the same address.

The function returns the old value.

5.4.5.1.4 atomicDec()

```
unsigned atomicDec(unsigned* address, unsigned val);
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes `(old == 0 || old > val) ? val : (old - 1)`.
3. Stores the result back to memory at the same address.

The function returns the old value.

5.4.5.1.5 atomicAnd()

```
T atomicAnd(T* address, T val);
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes `old & val`.
3. Stores the result back to memory at the same address.

The function returns the old value.

`atomicAnd()` supports the following data types:

- `int, unsigned, unsigned long long`.

5.4.5.1.6 atomicOr()

```
T atomicOr(T* address, T val);
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes `old | val`.
3. Stores the result back to memory at the same address.

The function returns the old value.

`atomicOr()` supports the following data types:

- `int, unsigned, unsigned long long`.

5.4.5.1.7 atomicXor()

```
T atomicXor(T* address, T val);
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes `old ^ val`.
3. Stores the result back to memory at the same address.

The function returns the old value.

`atomicXor()` supports the following data types:

- `int`, `unsigned`, `unsigned long long`.

5.4.5.1.8 `atomicMin()`

```
T atomicMin(T* address, T val);
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes the minimum of old and `val`.
3. Stores the result back to memory at the same address.

The function returns the old value.

`atomicMin()` supports the following data types:

- `int`, `unsigned`, `unsigned long long`, `long long`.

5.4.5.1.9 `atomicMax()`

```
T atomicMax(T* address, T val);
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes the maximum of old and `val`.
3. Stores the result back to memory at the same address.

The function returns the old value.

`atomicMax()` supports the following data types:

- `int`, `unsigned`, `unsigned long long`, `long long`.

5.4.5.1.10 `atomicExch()`

```
T atomicExch(T* address, T val);
```

```
template<typename T>
T atomicExch(T* address, T val); // only 128-bit types, compute capability 9.x
↪ and higher
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Stores `val` back to memory at the same address.

The function returns the old value.

`atomicExch()` supports the following data types:

- `int`, `unsigned`, `unsigned long long`, `float`.

The C++ template function `atomicExch()` supports 128-bit types with the following requirements:

- ▶ Compute capability 9.x and higher.
- ▶ T must be aligned to 16 bytes, namely `alignof(T) >= 16`.
- ▶ T must be trivially copyable, namely `std::is_trivially_copyable_v<T>`.
- ▶ For C++03 and older: T must be trivially constructible, namely `std::is_default_constructible_v<T>`.

5.4.5.1.11 `atomicCAS()`

```
T atomicCAS(T* address, T compare, T val);
```

```
template<typename T>
T atomicCAS(T* address, T compare, T val); // only 128-bit types, compute
↳ capability 9.x and higher
```

The function performs the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Computes `old == compare ? val : old`.
3. Stores the result back to memory at the same address.

The function returns the old value.

`atomicCAS()` supports the following data types:

- ▶ `int`, `unsigned`, `unsigned long long`, `unsigned short`.

The C++ template function `atomicCAS()` supports 128-bit types with the following requirements:

- ▶ Compute capability 9.x and higher.
- ▶ T must be aligned to 16 bytes, namely `alignof(T) >= 16`.
- ▶ T must be trivially copyable, namely `std::is_trivially_copyable_v<T>`.
- ▶ For C++03 and older: T must be trivially constructible, namely `std::is_default_constructible_v<T>`.

5.4.5.2 Built-in Atomic Functions

CUDA 12.8 and later support CUDA compiler built-in functions for atomic operations, following the same memory ordering semantics as [C++ standard atomic operations](#) and the [CUDA thread scopes](#). The functions follow the [GNU's atomic built-in function signature](#) with an extra argument for thread scope.

`nvcc` defines the macro `__CUDACC_DEVICE_ATOMIC_BUILTINS__` when built-in atomic functions are supported.

Below are listed the raw enumerators for the [memory orders](#) and [thread scopes](#), which are used as the order and scope arguments of the built-in atomic functions:

```
// atomic memory orders
enum {
    __NV_ATOMIC_RELAXED,
    __NV_ATOMIC_CONSUME,
```

(continues on next page)

(continued from previous page)

```

__NV_ATOMIC_ACQUIRE,
__NV_ATOMIC_RELEASE,
__NV_ATOMIC_ACQ_REL,
__NV_ATOMIC_SEQ_CST
};

```

```

// thread scopes
enum {
    __NV_THREAD_SCOPE_THREAD,
    __NV_THREAD_SCOPE_BLOCK,
    __NV_THREAD_SCOPE_CLUSTER,
    __NV_THREAD_SCOPE_DEVICE,
    __NV_THREAD_SCOPE_SYSTEM
};

```

- ▶ The memory order corresponds to C++ standard atomic operations' memory order.
- ▶ The thread scope follows the `cuda::thread_scope` definition.
- ▶ `__NV_ATOMIC_CONSUME` memory order is currently implemented using stronger `__NV_ATOMIC_ACQUIRE` memory order.
- ▶ `__NV_THREAD_SCOPE_THREAD` thread scope is currently implemented using wider `__NV_THREAD_SCOPE_BLOCK` thread scope.

Example:

```

__device__ T __nv_atomic_load_n(T* pointer,
                                int memory_order,
                                int thread_scope = __NV_THREAD_SCOPE_SYSTEM);

```

Atomic built-in functions have the following restrictions:

- ▶ They can only be used in device functions.
- ▶ They cannot operate on local memory.
- ▶ The addresses of these functions cannot be taken.
- ▶ The order and scope arguments must be integer literals; they cannot be variables.
- ▶ The thread scope `__NV_THREAD_SCOPE_CLUSTER` is supported on architectures `sm_90` and higher.

Example of unsupported cases:

```

// Not permitted in a host function
__host__ void bar() {
    unsigned u1 = 1, u2 = 2;
    __nv_atomic_load(&u1, &u2, __NV_ATOMIC_RELAXED, __NV_THREAD_SCOPE_
SYSTEM);
}

// Not permitted to be applied to local memory
__device__ void foo() {
    unsigned a = 1, b;
    __nv_atomic_load(&a, &b, __NV_ATOMIC_RELAXED, __NV_THREAD_SCOPE_SYSTEM);
}

```

(continues on next page)

(continued from previous page)

```

}

// Not permitted as a template default argument.
// The function address cannot be taken.
template<void *F = __nv_atomic_load_n>
class X {
    void *f = F; // The function address cannot be taken.
};

// Not permitted to be called in a constructor initialization list.
class Y {
    int a;
public:
    __device__ Y(int *b): a(__nv_atomic_load_n(b, __NV_ATOMIC_RELAXED)) {}
};

```

5.4.5.2.1 __nv_atomic_fetch_add(), __nv_atomic_add()

```

__device__ T    __nv_atomic_fetch_add(T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);
__device__ void __nv_atomic_add      (T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);

```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
 2. Computes `old + val`.
 3. Stores the result back to memory at the same address.
- `__nv_atomic_fetch_add` returns the old value.
 - `__nv_atomic_add` has no return value.

The functions support the following data types:

- `int`, `unsigned`, `unsigned long long`, `float`, `double`.

5.4.5.2.2 __nv_atomic_fetch_sub(), __nv_atomic_sub()

```

__device__ T    __nv_atomic_fetch_sub(T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);
__device__ void __nv_atomic_sub      (T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);

```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
 2. Computes `old - val`.
 3. Stores the result back to memory at the same address.
- `__nv_atomic_fetch_sub` returns the old value.
 - `__nv_atomic_sub` has no return value.

The functions support the following data types:

- `int`, `unsigned`, `unsigned long long`, `float`, `double`.

5.4.5.2.3 `__nv_atomic_fetch_and()`, `__nv_atomic_and()`

```
__device__ T    __nv_atomic_fetch_and(T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);
__device__ void __nv_atomic_and      (T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);
```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
 2. Computes `old & val`.
 3. Stores the result back to memory at the same address.
- `__nv_atomic_fetch_and` returns the old value.
 - `__nv_atomic_and` has no return value.

The functions support the following data types:

- Any integral type of size 4 or 8 bytes.

5.4.5.2.4 `__nv_atomic_fetch_or()`, `__nv_atomic_or()`

```
__device__ T    __nv_atomic_fetch_or(T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);
__device__ void __nv_atomic_or      (T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);
```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
 2. Computes `old | val`.
 3. Stores the result back to memory at the same address.
- `__nv_atomic_fetch_or` returns the old value.
 - `__nv_atomic_or` has no return value.

The functions support the following data types:

- Any integral type of size 4 or 8 bytes.

5.4.5.2.5 `__nv_atomic_fetch_xor()`, `__nv_atomic_xor()`

```
__device__ T    __nv_atomic_fetch_xor(T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);
__device__ void __nv_atomic_xor      (T* address, T val, int order, int scope
↳= __NV_THREAD_SCOPE_SYSTEM);
```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.

2. Computes $\text{old} \wedge \text{val}$.
 3. Stores the result back to memory at the same address.
- `__nv_atomic_fetch_xor` returns the old value.
 - `__nv_atomic_xor` has no return value.

The functions support the following data types:

- Any integral type of size 4 or 8 bytes.

5.4.5.2.6 `__nv_atomic_fetch_min()`, `__nv_atomic_min()`

```
__device__ T __nv_atomic_fetch_min(T* address, T val, int order, int scope
↪ = __NV_THREAD_SCOPE_SYSTEM);
__device__ void __nv_atomic_min      (T* address, T val, int order, int scope
↪ = __NV_THREAD_SCOPE_SYSTEM);
```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
 2. Computes the minimum of `old` and `val`.
 3. Stores the result back to memory at the same address.
- `__nv_atomic_fetch_min` returns the old value.
 - `__nv_atomic_min` has no return value.

The functions support the following data types:

- `unsigned`, `int`, `unsigned long long`, `long long`.

5.4.5.2.7 `__nv_atomic_fetch_max()`, `__nv_atomic_max()`

```
__device__ T __nv_atomic_fetch_max(T* address, T val, int order, int scope
↪ = __NV_THREAD_SCOPE_SYSTEM);
__device__ void __nv_atomic_max      (T* address, T val, int order, int scope
↪ = __NV_THREAD_SCOPE_SYSTEM);
```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
 2. Computes the maximum of `old` and `val`.
 3. Stores the result back to memory at the same address.
- `__nv_atomic_fetch_max` returns the old value.
 - `__nv_atomic_max` has no return value.

The functions support the following data types:

- `unsigned`, `int`, `unsigned long long`, `long long`

5.4.5.2.8 `__nv_atomic_exchange()`, `__nv_atomic_exchange_n()`

```

__device__ T    __nv_atomic_exchange_n(T* address, T val,          int order,
↪int scope = __NV_THREAD_SCOPE_SYSTEM);
__device__ void __nv_atomic_exchange (T* address, T* val, T* ret, int order,
↪int scope = __NV_THREAD_SCOPE_SYSTEM);

```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
 2. `__nv_atomic_exchange_n` stores `val` to where `address` points to.
`__nv_atomic_exchange` stores old to where `ret` points to and stores the value located at the address `val` to where `address` points to.
- `__nv_atomic_exchange_n` returns the old value.
 - `__nv_atomic_exchange` has no return value.

The functions support the following data types:

- Any data type of size of 4, 8 or 16 bytes.
- The 16-byte data type is supported on devices of compute capability 9.x and higher.

5.4.5.2.9 `__nv_atomic_compare_exchange()`, `__nv_atomic_compare_exchange_n()`

```

__device__ bool __nv_atomic_compare_exchange (T* address, T* expected, T*
↪desired, bool weak, int success_order, int failure_order,
                                         int scope = __NV_THREAD_SCOPE_
↪SYSTEM);
__device__ bool __nv_atomic_compare_exchange_n(T* address, T* expected, T
↪desired, bool weak, int success_order, int failure_order,
                                         int scope = __NV_THREAD_SCOPE_
↪SYSTEM);

```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. Compare old with the value where `expected` points to.
3. If they are equal, the return value is `true` and `desired` is stored to where `address` points to. Otherwise, it returns `false` and old is stored to where `expected` points to.

The parameter `weak` is ignored and it picks the stronger memory order between `success_order` and `failure_order` to execute the compare-and-exchange operation.

The functions support the following data types:

- Any data type of size of 2, 4, 8 or 16 bytes.
- The 16-byte data type is supported on devices with compute capability 9.x and higher.

5.4.5.2.10 `__nv_atomic_load()`, `__nv_atomic_load_n()`

```
__device__ void __nv_atomic_load (T* address, T* ret, int order, int scope =
↳ __NV_THREAD_SCOPE_SYSTEM);
__device__ T __nv_atomic_load_n(T* address, int order, int scope =
↳ __NV_THREAD_SCOPE_SYSTEM);
```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. `__nv_atomic_load` stores old to where `ret` points to.
`__nv_atomic_load_n` returns old.

The functions support the following data types:

- Any data type of size 1, 2, 4, 8 or 16 bytes.

order cannot be `__NV_ATOMIC_RELEASE` or `__NV_ATOMIC_ACQ_REL`.

5.4.5.2.11 `__nv_atomic_store()`, `__nv_atomic_store_n()`

```
__device__ void __nv_atomic_store (T* address, T* val, int order, int scope
↳ = __NV_THREAD_SCOPE_SYSTEM);
__device__ void __nv_atomic_store_n(T* address, T val, int order, int scope
↳ = __NV_THREAD_SCOPE_SYSTEM);
```

The functions perform the following operations in one atomic transaction:

1. Reads the old value located at the address `address` in global or shared memory.
2. `__nv_atomic_store` reads the value where `val` points to and stores to where `address` points to.
`__nv_atomic_store_n` stores `val` to where `address` points to.

order cannot be `__NV_ATOMIC_CONSUME`, `__NV_ATOMIC_ACQUIRE` or `__NV_ATOMIC_ACQ_REL`.

5.4.5.2.12 `__nv_atomic_thread_fence()`

```
__device__ void __nv_atomic_thread_fence(int order, int scope = __NV_THREAD_
↳ SCOPE_SYSTEM);
```

This atomic function establishes an ordering between memory accesses requested by this thread based on the specified memory order. The thread scope parameter specifies the set of threads that may observe the ordering effect of this operation.

5.4.6. Warp Functions

The following section describes the warp functions that allow threads within a warp to communicate with each other and perform computations.

Hint

It is suggested to use the CUB [Warp-Wide “Collective” Primitives](#) to perform warp operations whenever possible for efficiency, safety, and portability reasons.

5.4.6.1 Warp Active Mask

```
unsigned __activemask();
```

The function returns a 32-bit integer mask representing all currently active threads in the calling warp. The Nth bit is set if the Nth lane in the warp is active when `__activemask()` is called. *Inactive threads* are represented by 0 bits in the returned mask. Threads that have exited the program are always marked as inactive.

Warning

`__activemask()` cannot be used to determine which warp lanes execute a given branch. This function is intended for opportunistic warp-level programming and only provides an instantaneous snapshot of the active threads within a warp.

```
// Check whether at least one thread's predicate evaluates to true
if (pred) {
    // Invalid: the value of 'at_least_one' is non-deterministic
    // and could vary between executions.
    at_least_one = __activemask() > 0;
}
```

Note that threads convergent at an `__activemask()` call are not guaranteed to remain convergent at subsequent instructions unless those instructions are warp synchronizing intrinsics (`__sync`).

For example, the compiler could reorder instructions, and the set of active threads might not be preserved:

```
unsigned mask      = __activemask();           // Assume mask ==
    → 0xFFFFFFFF (all bits set, all threads active)
int      predicate = threadIdx.x % 2 == 0;     // 1 for even threads, 0
    → for odd threads
int      result    = __any_sync(mask, predicate); // Active threads might not
    → be preserved
```

5.4.6.2 Warp Vote Functions

```
int      __all_sync  (unsigned mask, int predicate);
int      __any_sync  (unsigned mask, int predicate);
unsigned __ballot_sync(unsigned mask, int predicate);
```

The warp vote functions enable the threads of a given *warp* to perform a reduction-and-broadcast operation. These functions take an integer predicate as input from each non-exited thread in the warp and compare those values with zero. The results of the comparisons are then combined (reduced) across the *active threads* of the warp in one of the following ways, broadcasting a single return value to each participating thread:

__all_sync(unsigned mask, predicate):

Evaluates predicate for all non-exited threads in mask and returns non-zero if predicate evaluates to non-zero for all of them.

__any_sync(unsigned mask, predicate):

Evaluates predicate for all non-exited threads in mask and returns non-zero if predicate evaluates to non-zero for one or more of them.

__ballot_sync(unsigned mask, predicate):

Evaluates predicate for all non-exited threads in mask and returns an integer whose Nth bit is set if predicate evaluates to non-zero for the Nth thread of the warp and the Nth thread is active. Otherwise, the Nth bit is zero.

The functions are subject to the *Warp __sync Intrinsic Constraints*.

Warning

These intrinsics do not provide any memory ordering.

5.4.6.3 Warp Match Functions**Hint**

It is suggested to use the `libcu++ cuda::device::warp_match_all()` function as a generalized and safer alternative to `__match_all_sync` function.

```
unsigned __match_any_sync(unsigned mask, T value);
unsigned __match_all_sync(unsigned mask, T value, int *pred);
```

The warp match functions perform a broadcast-and-compare operation of a variable between non-exited threads within a *warp*.

__match_any_sync

Returns the mask of non-exited threads that have the same bitwise value in mask.

__match_all_sync

Returns mask if all non-exited threads in mask have the same bitwise value; otherwise 0 is returned. Predicate pred is set to true if all non-exited threads in mask have the same bitwise value; otherwise the predicate is set to false.

T can be `int`, `unsigned`, `long`, `unsigned long`, `long long`, `unsigned long long`, `float` or `double`.

The functions are subject to the *Warp __sync Intrinsic Constraints*.

Warning

These intrinsics do not provide any memory ordering.

5.4.6.4 Warp Reduce Functions**Hint**

It is suggested to use the CUB *Warp-Wide “Collective” Primitives* to perform a Warp Reduction whenever possible for efficiency, safety, and portability reasons.

Supported by devices of compute capability 8.x or higher.

```

T    __reduce_add_sync(unsigned mask, T value);
T    __reduce_min_sync(unsigned mask, T value);
T    __reduce_max_sync(unsigned mask, T value);

unsigned __reduce_and_sync(unsigned mask, unsigned value);
unsigned __reduce_or_sync (unsigned mask, unsigned value);
unsigned __reduce_xor_sync(unsigned mask, unsigned value);

```

The `__reduce_<op>_sync` intrinsics perform a reduction operation on the data provided in `value` after synchronizing all non-exited threads named in `mask`.

__reduce_add_sync, __reduce_min_sync, __reduce_max_sync

Returns the result of applying an arithmetic add, min, or max reduction operation on the values provided in `value` by each non-exited thread named in `mask`. `T` can be an unsigned or signed integer.

__reduce_and_sync, __reduce_or_sync, __reduce_xor_sync

Returns the result of applying a bitwise AND, OR, or XOR reduction operation on the values provided in `value` by each non-exited thread named in `mask`.

The functions are subject to the *Warp __sync Intrinsic Constraints*.

Warning

These intrinsics do not provide any memory ordering.

5.4.6.5 Warp Shuffle Functions

Hint

It is suggested to use the `libcu++` `cuda::device::warp_shuffle()` functions as a generalized and safer alternative to `__shfl_sync()` and `__shfl_<op>_sync()` intrinsics.

```

T __shfl_sync      (unsigned mask, T value, int      srcLane, int
↳width=warpSize);
T __shfl_up_sync   (unsigned mask, T value, unsigned delta,  int
↳width=warpSize);
T __shfl_down_sync(unsigned mask, T value, unsigned delta,  int
↳width=warpSize);
T __shfl_xor_sync  (unsigned mask, T value, int      laneMask, int
↳width=warpSize);

```

Warp shuffle functions exchange a value between non-exited threads within a *warp* without the use of shared memory.

__shfl_sync(): Direct copy from indexed lane.

The intrinsic function returns the value of `value` held by the thread whose ID is given by `srcLane`.

- ▶ If `width` is less than `warpSize`, then each subsection of the warp behaves as a separate entity with a starting logical lane ID of 0.
- ▶ If `srcLane` is outside the range `[0, width - 1]`, the result corresponds to the value held by the `srcLane % width`, which is within the same subsection.